



Frankfurt University of Applied Sciences

Fachbereich 2: Informatik und Ingenieurwissenschaften

Masterarbeit

Explorative Untersuchung einer lokalen Vergleichspipeline mit Smartphoneanregung

Thema:

KI-basierte Anomalieerkennung auf ressourcenbeschränkter Edge-Hardware

Erster Prüfer: Herr Prof. Dr. Karsten Schmidt

Zweite Prüferin: Frau M.Sc. Virginia Kramer

Studiengang: Mechatronik und Robotik

Bearbeiter: Abdulmalik Alzubi

Matrikelnummer: 1224063

Zeitraum: 26.05.2026 – 27.10.2026

Abgabedatum: 27.10.2026

Erklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbständig erstellt und keine anderen als die angegebenen Hilfsmittel benutzt habe.

Soweit ich auf fremde Materialien, Texte oder Gedankengänge zurückgegriffen habe, enthalten meine Ausführungen vollständige und eindeutige Verweise auf die Urheber und Quellen.

Alle weiteren Inhalte der vorgelegten Arbeit stammen von mir im urheberrechtlichen Sinn, soweit keine Verweise und Zitate erfolgen.

Datum: Frankfurt, den _____

Unterschrift: _____

Abdulmalik Alzubi

Zusammenfassung

Diese Arbeit entwickelt und untersucht eine lokale Vergleichspipeline für die Anomalieerkennung aus dreiachsigen Beschleunigungsdaten. Ein Raspberry Pi 5 verarbeitet die Messungen eines ADXL345 an einem Lüfteraufbau mit RMS-Schwelle, Isolation Forest und einem Float32-TFLite-Autoencoder. Gegenstand ist eine explorative Pilotstudie mit äußerer mechanischer Anregung durch ein vibrierendes Smartphone auf dem Tisch. Alle Verfahren erhalten nicht überlappende Fenster aus 128 XYZ-Punkten und verwenden ein vor den Smartphoneaufnahmen erstelltes, eingefrorenes Normalprofil.

Die Untersuchung vom 19. September 2026 umfasst drei ausgewählte Einzelereignisse, deren Offline-Replay und einen gemeinsamen Live-Lauf. Im gemeinsamen Lauf markieren Autoencoder und Isolation Forest erstmals dasselbe Fenster 108, RMS erst Fenster 109. Die Autoencoder-Ausgabe liegt etwa 19,1 ms vor Isolation Forest und 615,7 ms vor RMS. Im Replay der drei Einzelereignisse markiert der Autoencoder das erste Alarmfenster einmal früher und zweimal gleichzeitig mit den Baselines. Diese Beobachtungen beschreiben wenige Ereignisse und belegen keine allgemeine Überlegenheit.

Der gemeinsame Prozess benötigt einen CPU-Median von 13,04 % eines logischen Kerns und maximal 174,42 MiB abgetasteten RSS. Bei 196 überprüfbaren Folgefensterankünften je Methode treten keine Fristüberschreitungen auf. Die lokale Ausführbarkeit ist damit für den beobachteten Betrieb belegt. Die Aussagekraft zur Erkennung begrenzen wenige unabhängige Ereignisse, eine Kalibrierungsaufnahme mit nur 97 Fenstern und fehlende physikalische Zeitreferenzen der Smartphoneanregung. Deren absolute Erkennungsverzögerung, F1 und bestätigte Fehlalarmrate sind deshalb nicht bestimmbar. Elektrische Leistung und Energie wurden nicht gemessen. Der Beitrag besteht in der instrumentierten Vergleichspipeline und der nachvollziehbaren Bewertung ihrer technischen Leistungsfähigkeit und empirischen Grenzen.

Inhaltsverzeichnis

Erklärung.....	i
Zusammenfassung.....	ii
Abbildungsverzeichnis.....	v
Tabellenverzeichnis.....	vi
Abkürzungsverzeichnis.....	vii
1 Einleitung, Problemstellung und Zielsetzung.....	1
1.1 Ausgangssituation und Motivation.....	1
1.2 Wissenschaftliche Problemstellung.....	1
1.3 Zielsetzung der Arbeit.....	1
1.4 Forschungsfragen und Bewertungsmaßstab.....	2
1.5 Abgrenzung der Arbeit.....	2
1.6 Aufbau der Arbeit.....	3
2 Theoretische Grundlagen.....	4
2.1 Zustandsüberwachung und Anomalieerkennung.....	4
2.2 Verfahrensklassen für die Anomalieentscheidung.....	4
2.2.1 Statistische Schwellenwertverfahren.....	4
2.2.2 Klassische Verfahren im Normaldaten-Szenario.....	4
2.2.3 Autoencoder.....	5
2.3 Messdaten und Fensterbildung.....	5
2.4 Datenaufteilung und Vorverarbeitung.....	5
2.5 Schwellenwertkalibrierung.....	6
2.6 Erkennungsqualität und Aussagekraft der Evaluation.....	6
2.7 Lokale Ausführung und Zeitverhalten.....	7
2.8 Veränderte Normalbedingungen.....	8
3 Stand der Technik und verwandte Arbeiten.....	9
3.1 Datenbasis und Messbedingungen.....	9
3.2 Verfahren und Edge-Ausführung.....	9
3.3 Vergleich ausgewählter Arbeiten.....	10
3.4 Forschungslücke und Einordnung der eigenen Arbeit.....	10
4 Anforderungsanalyse.....	12
4.1 Messdaten und Vergleichsbasis.....	12
4.2 Lokale Verarbeitung und Zeitverhalten.....	12
4.3 Evaluation.....	13
4.4 Kernanforderungen und Nachweis.....	13
5 Variantenvergleich und Systementscheidung.....	16
5.1 Bewertungsrahmen und Vorgehen.....	16
5.2 Bewertung der Prüfstandsalternativen.....	17
5.3 Auswahl von Sensorik und Edge-Plattform.....	19
5.4 Datenbasis, Signalrepräsentation und Verfahren.....	22
5.5 Kriterienbasierte Bewertung der Softwarealternativen.....	23
5.6 Systementscheidung und Übergang zur Implementierung.....	26
6 Implementierung und Versuchsdesign.....	27
6.1 Smartphoneversuch und Untersuchungsziele.....	27

6.2 Hardware und mechanischer Übertragungsweg.....	27
6.3 PWM, Sensor und Zeitbasis.....	28
6.4 Vorverarbeitung und eingefrorene Modelle.....	29
6.5 Normaltraining, Schwellen und Exportprüfung.....	30
6.6 Einzelbetrieb, Replay und gemeinsame Live-Auswertung.....	33
6.7 Ablauf, Bedienmarker und Annahmen.....	34
6.8 Dateien, Visualisierung und Nachvollziehbarkeit.....	35
7 Evaluation.....	36
7.1 Umfang und Auswahl der Aufnahmen.....	36
7.2 Datenintegrität und Reproduzierbarkeit.....	36
7.3 Einzelbetrieb und Vergleich auf identischen Replay-Fenstern.....	37
7.4 Gemeinsamer Live-Lauf: Signal und Alarmreihenfolge.....	38
7.5 Warum unterscheiden sich die ersten Alarme?.....	40
7.6 Rechenaufwand und Verhalten des Raspberry Pi.....	41
7.7 Forschungsfragen und Nachweisstand.....	45
8 Diskussion.....	47
8.1 Alarmverhalten und Ausgabezeitpunkte.....	47
8.2 Schwellen, Alarmhäufigkeit und faire Bewertung.....	47
8.3 Vergleichbarkeit und interne Gültigkeit.....	48
8.4 Raspberry-Pi-Bedarf und Grenzen der Übertragung.....	48
9 Ausblick.....	49
10 Fazit.....	50
Anhang.....	51
Digitale Belege und Fassungszuordnung.....	51
Lesen der aktuellen Messdateien.....	52
KI-Unterstützung und fachliche Verantwortung.....	52
Literaturverzeichnis.....	53

Abbildungsverzeichnis

Abbildung 6-1: Schematischer Aufbau des Smartphoneversuchs.....	28
Abbildung 6-2: Trainings- und Validierungsverlust des Autoencoders.....	31
Abbildung 6-3: Verteilung der Kalibrierungsscores und eingefrorene P99-Schwellen.....	32
Abbildung 6-4: Datenfluss des gemeinsamen Live-Vergleichs.....	33
Abbildung 7-1: Erstes Alarmfenster im Offline-Replay.....	38
Abbildung 7-2: Gemeinsamer Rohdaten- und Scoreverlauf.....	39
Abbildung 7-3: Nummerierte Reihenfolge der ersten Alarmausgaben im gemeinsamen Lauf und zugehörige Fensterindizes.....	40
Abbildung 7-4: Schwellenübergang der Fenster 107 bis 109.....	41
Abbildung 7-5: Verarbeitungszeiten im Einzelbetrieb und im gemeinsamen Prozess.....	42
Abbildung 7-6: CPU und RAM im Einzelbetrieb und gemeinsamen Live-Prozess.....	43
Abbildung 7-7: Zeitlicher Ressourcenverlauf des gemeinsamen Prozesses.....	44

Tabellenverzeichnis

Tabelle 2-1: Kennzahlen der binären Anomalieerkennung.....	7
Tabelle 3-1: Ausgewählte Vorarbeiten und ihre Bedeutung für die Untersuchung.....	10
Tabelle 4-1: Anforderungen, Prioritäten und Nachweise.....	14
Tabelle 5-1: Bewertung der Prüfstandsalternativen.....	17
Tabelle 5-2: Kriterien und Gewichte für den Prüfstand.....	18
Tabelle 5-3: Gewichtete Bewertung der Prüfstandsalternativen.....	18
Tabelle 5-4: Vergleich der Sensorik und Messketten.....	19
Tabelle 5-5: Bewertung der Edge-Plattformen.....	20
Tabelle 5-6: Kriterien und Gewichte für die Edge-Plattform.....	20
Tabelle 5-7: Gewichtete Bewertung der Edge-Plattformen.....	21
Tabelle 5-8: Bewertung der Signalrepräsentationen.....	22
Tabelle 5-9: Auswahl der Anomalieverfahren.....	23
Tabelle 5-10: Kriterienbasierter Vergleich der Softwarealternativen.....	24
Tabelle 5-11: Kriterien und Gewichte für die Softwareauswahl.....	24
Tabelle 5-12: Gewichtete Bewertung der Softwarealternativen.....	25
Tabelle 5-13: Systementscheidungen und ausstehende Nachweise.....	26
Tabelle 6-1: Hardware und Betriebszustand der Smartphoneuntersuchung.....	28
Tabelle 6-2: Festgelegte Skalierung des aktuellen Normalprofils, gerundet.....	29
Tabelle 6-3: Architektur des verwendeten Autoencoders.....	30
Tabelle 6-4: Eingefrorene Schwellen der Smartphoneuntersuchung.....	32
Tabelle 6-5: Zeitbegriffe und zulässige Interpretation.....	34
Tabelle 6-6: Annahmen und offene Nachweise.....	35
Tabelle 7-1: Erhaltene Anläufe und Auswertungsrolle.....	36
Tabelle 7-2: Ausgewählte Aufnahmen und Verteilung der gültigen Fenster.....	37
Tabelle 7-3: Replay derselben Rohfenster; erstes Alarmfenster und gesamte Alarmfenster.....	38
Tabelle 7-4: Erste Alarme im gemeinsamen Live-Lauf.....	39
Tabelle 7-5: Unmittelbarer Schwellenvergleich auf dem gemeinsamen Fenster 108.....	40
Tabelle 7-6: Verarbeitungszeit im jeweiligen Auswerteraufruf; ms.....	42
Tabelle 7-7: Prozessressourcen der ausgewählten Smartphoneaufnahmen.....	43
Tabelle 7-8: Beantwortung der Forschungsfragen und Nachweisgrenzen.....	45
Tabelle 7-9: Rückbindung an die Anforderungen aus Kapitel 4.....	46
Tabelle A-1: Nachweisverzeichnis.....	51

Abkürzungsverzeichnis

Abkürzung	Bedeutung
AC	Wechselanteil nach Entfernung des Gleichanteils
AE	Autoencoder
CPU	Central Processing Unit
DFT	Diskrete Fourier-Transformation
FFT	Fast Fourier Transform
FPR	False-Positive-Rate
GPIO	General Purpose Input/Output
GUI	Graphical User Interface (grafische Benutzeroberfläche)
IEPE	Integrated Electronics Piezo-Electric
IF	Isolation Forest
I ² C	Inter-Integrated Circuit
KI	Künstliche Intelligenz
LSB	Least Significant Bit
MCU	Microcontroller Unit
MiB	Mebibyte (2 ²⁰ Byte)
ML	Machine Learning
MSE	Mean Squared Error
ODR	Output Data Rate
P99	Empirisches 99. Perzentil
PSD	Power Spectral Density (Leistungsdichtespektrum)
PWM	Pulsweitenmodulation
RAM	Random Access Memory
RMS	Root Mean Square
RSS	Resident Set Size
SBC	Single-Board Computer
SPI	Serial Peripheral Interface
TFLite	TensorFlow Lite

1 Einleitung, Problemstellung und Zielsetzung

1.1 Ausgangssituation und Motivation

Beschleunigungssignale ermöglichen es, Schwingungen technischer Systeme zu beobachten. Bei rotierenden Komponenten können Veränderungen dieser Signale auf einen veränderten Zustand hinweisen. Randall und Antoni (2011) erläutern dies am Beispiel der Wälzlagerdiagnose. Die vorliegende Arbeit untersucht die lokale Erkennung solcher Abweichungen anhand von Beschleunigungsdaten.

Die Auswertung soll lokal in der Nähe des Sensors erfolgen. Dieses Prinzip des Edge Computing kann die Übertragung von Rohdaten und die Abhängigkeit von entfernten Rechendiensten verringern (Shi et al., 2016). Neben dem beobachteten Alarmverhalten sind deshalb der Rechen- und Speicherbedarf sowie die Verarbeitungszeit wichtig. Die vorliegende Pilotstudie untersucht diese Größen bei einer zusätzlichen äußeren Smartphoneanregung an einem laufenden Lüfteraufbau.

1.2 Wissenschaftliche Problemstellung

Bei der normalitätsbasierten Anomalieerkennung wird ein Referenzmodell aus normalen Messdaten gebildet. Neue Daten gelten als auffällig, wenn sie ausreichend von dieser Referenz abweichen. Dieses Vorgehen benötigt keine vollständige Sammlung möglicher Fehlerbeispiele. Seine Aussagekraft hängt jedoch davon ab, welche Normalbedingungen die Referenzdaten abdecken (Chandola et al., 2009).

Für einen belastbaren Vergleich müssen die Verfahren unter vergleichbaren Bedingungen bewertet werden. Informationen aus dem Test dürfen weder die Modellwahl noch die Vorverarbeitung oder die Entscheidungsschwelle beeinflussen; andernfalls kann die Erkennungsleistung überschätzt werden (Kaufman et al., 2012). Außerdem muss die gesamte Verarbeitung auf der Zielhardware betrachtet werden. Eine kurze Modellinferenz allein zeigt noch nicht, ob alle eingehenden Daten rechtzeitig verarbeitet werden.

1.3 Zielsetzung der Arbeit

Ziel ist die Konzeption, Umsetzung und experimentelle Bewertung einer lokalen Verarbeitungskette (Pipeline) für mehrkanalige Beschleunigungsdaten. Verglichen werden ein neuronales Hauptverfahren, eine statistische Schwellenwertbaseline und ein klassisches Lernverfahren. Auch das klassische Verfahren gehört zum maschinellen Lernen. Die konkrete Auswahl des Systems und des Versuchsaufbaus wird in Kapitel 5 anhand der Anforderungen und eines Variantenvergleichs begründet.

Der Beitrag ist eine instrumentierte Vergleichspipeline, die identische Sensoreingaben, nachvollziehbare Schwellenkalibrierung und Messungen des technischen Aufwands verbindet. Die explorative Smartphoneuntersuchung analysiert Alarmfenster und Ausgabezeitpunkte in Einzelbetrieb, Replay und gemeinsamem Live-Betrieb. Sie bewertet die

technische Ausführbarkeit und legt offen, welche Aussagen durch die vorhandenen Messungen gedeckt sind. Die zusätzliche Tischanregung wird nicht als nachgewiesener Lüfter- oder Lagerschaden interpretiert.

1.4 Forschungsfragen und Bewertungsmaßstab

Zentrale Forschungsfrage: Wie lässt sich eine lokale Anomalieerkennung für dreiachsige Beschleunigungsdaten auf Edge-Hardware umsetzen, und welches Alarmverhalten, Zeitverhalten und welcher Ressourcenbedarf sind bei einer äußeren Smartphoneanregung im untersuchten Pilotaufbau beobachtbar?

FF1: Welche Anforderungen ergeben sich aus der Aufgabe, und wie begründen sie die Auswahl von Versuchsaufbau, Sensorik, Hardware und Verfahren?

FF2: Wie unterscheiden sich Autoencoder, RMS und Isolation Forest bei identischen Testfenstern und einer gemeinsamen Kalibrierungsregel hinsichtlich Alarmzahl, erstem Alarmfenster und relativem Zeitpunkt der ersten Alarmausgabe?

FF3: Wie viel Speicher und Rechenleistung benötigt die lokale Pipeline, und wie häufig überschreitet ihre Verarbeitung das Zeitbudget bis zum nächsten Fenster?

FF4: In welchem Umfang sichern Datenintegrität, dokumentierter Aufbau, Ereignisreferenz und Wiederholungen die Vergleichbarkeit und Aussagekraft der beobachteten Ergebnisse ab?

Die Forschungsfragen wurden im Projektverlauf auf den hier untersuchten Smartphoneversuch ausgerichtet. Sie strukturieren eine explorative, deskriptive Evaluation und sind keine vorab registrierten statistischen Hypothesen. Training und Schwellenkalibrierung verwenden vorher aufgenommene Normaldaten; die eingefrorenen Schwellen werden nicht an den Smartphoneaufnahmen angepasst. Ohne unabhängige physikalische Ereignislabels werden weder eine höhere F1-Leistung noch eine allgemein kürzere Erkennungsverzögerung behauptet.

Als technischer Bewertungsmaßstab für FF3 gilt: Das empirische 99. Perzentil (P_{99}) der Verarbeitungslatenz soll unter dem Fensterintervall S/f_s liegen. Die Latenz reicht vom vollständigen Fenster bis zur verfügbaren Entscheidung. S bezeichnet die Schrittweite in Messzeitpunkten, f_s die aus Host-Zeitstempeln ermittelte Erfassungsrate. Zusätzlich wird für jede überprüfbare Entscheidung gezählt, ob sie vor der tatsächlichen Ankunft des nächsten Fensters vorliegt. Das Kriterium ist ein technisches Ausführungsziel; P_{99} garantiert keine fristgerechte Ausgabe jeder Entscheidung.

1.5 Abgrenzung der Arbeit

Die Ergebnisse gelten für den dokumentierten Smartphoneversuch am Lüfteraufbau. Untersucht wird die Reaktion auf eine zusätzliche äußere mechanische Anregung; ein realer Maschinenfehler wird nicht nachgebildet oder diagnostisch bestätigt. Die Ergebnisse

erlauben weder eine gesicherte Schadensdiagnose noch eine Prognose der Restlebensdauer. Andere Systeme und industrieller Dauerbetrieb erfordern eigene Untersuchungen.

Der Kernversuch bewertet die lokale Inferenz eines zuvor trainierten Modells auf einer Zielplattform. Die Pipeline bleibt dabei unverändert. Eine mögliche spätere Nachkalibrierung wäre als eigener Versuch auszuwerten. Sie ist nicht Bestandteil des hier abgeschlossenen eingefrorenen Vergleichs.

Der untersuchte Datenstand umfasst die Smartphoneversuche vom 19. September 2026 mit dem Normalprofil vom 18. September. Der Lüfter läuft bei einer PWM-Vorgabe von 75 %. Nicht erhobene Drehzahlmessungen, eine fehlende unabhängige Zeitreferenz der Smartphonevibration und fehlende elektrische Verbrauchsmessungen begrenzen die Bewertung. Geänderte Betriebspunkte und eine Nachkalibrierung sind kein untersuchter Ergebnisgegenstand.

1.6 Aufbau der Arbeit

Kapitel 2 und 3 erläutern Grundlagen und verwandte Arbeiten. Kapitel 4 und 5 leiten Anforderungen ab und begründen die Systemauswahl. Kapitel 6 beschreibt den Smartphoneaufbau, die Implementierung und das Versuchsprotokoll. Kapitel 7 bewertet Alarmfenster, Ausgabezeitpunkte, Ressourcen und Nachweisgrenzen. Diskussion, Ausblick und Fazit beantworten die Forschungsfragen im Rahmen der vorhandenen Evidenz. Der Anhang dokumentiert digitale Belege, Softwareumgebung und die Interpretation der Messdateien.

2 Theoretische Grundlagen

Dieses Kapitel erklärt die benötigten Begriffe, mögliche Verfahren und die Messgrößen für die Evaluation. Die dargestellten Ansätze bilden die Grundlage für den Variantenvergleich; ihre Auswahl für das System erfolgt erst in Kapitel 5.

2.1 Zustandsüberwachung und Anomalieerkennung

Zustandsüberwachung (Condition Monitoring) erfasst und bewertet den technischen Zustand. Diagnose ordnet Fehlerursachen zu, Prognose schätzt die künftige Zustandsentwicklung (Jardine et al., 2006). Anomalieerkennung bewertet zunächst nur Abweichungen von einer Normalreferenz.

Was als anomal gilt, hängt von Referenz und Betriebskontext ab. Auch ein zeitlicher Verlauf kann auffällig sein, obwohl seine Einzelwerte unauffällig sind (Chandola et al., 2009). Deshalb werden hier Messfenster bewertet.

Klassische und neuronale Verfahren können auf als normal ausgewählten Daten angepasst werden. Dieses Normaldaten-Szenario beschreibt die Trainingsgrundlage und ist von der Algorithmusklasse zu unterscheiden. Der Anomalie-Score ist eine Bewertungsgröße, zunächst keine Schadenswahrscheinlichkeit (Ruff et al., 2021).

2.2 Verfahrensklassen für die Anomalieentscheidung

2.2.1 Statistische Schwellenwertverfahren

Der Effektivwert (Root Mean Square, RMS) eines Kanals mit N Messwerten ist die Quadratwurzel des Mittelwerts der quadrierten Werte:

$$RMS(x) = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} x[n]^2} \quad (2-1)$$

Der RMS fasst die Amplitude eines Fensters zusammen, erhält aber die Reihenfolge der Werte nicht. Für den Fenstermittelwert $\bar{x}$ gilt $RMS^2(x) = \bar{x}^2 + RMS^2(x - \bar{x})$. Ohne Mittelwertentfernung beeinflussen daher auch konstante Anteile den Kennwert. Beschleunigungssensoren erfassen neben Bewegung unter anderem die Gravitation und einen Sensoroffset (Analog Devices, 2022). Rohwert-RMS und RMS nach Mittelwertentfernung sind deshalb zu unterscheiden. Ebenso muss feststehen, wie mehrere Messkanäle zusammengefasst werden.

2.2.2 Klassische Verfahren im Normaldaten-Szenario

Isolation Forest nutzt zufällige Teilungsbäume. Jeder Schritt teilt die Daten anhand eines zufällig gewählten Merkmals und Teilungspunkts. Beobachtungen, die im Mittel durch wenige Teilungen isoliert werden, gelten als auffälliger (Liu et al., 2008). Im Normaldaten-Szenario erfolgt seine Anpassung ausschließlich anhand normaler Beobachtungen.

Die Eingabe ist ein Vektor fester Länge aus Fensterkennwerten oder aneinandergereihten Messwerten. Die verwendete Darstellung und die Richtung des ausgegebenen Scores müssen dokumentiert werden.

2.2.3 Autoencoder

Ein Autoencoder erzeugt aus einer Eingabe x eine Rekonstruktion $\hat{x}$. Der Encoder bildet die Eingabe auf eine interne Darstellung ab, der Decoder rekonstruiert daraus die Ausgangswerte. Eine begrenzte Modellkapazität oder Regularisierung soll bloßes Kopieren verhindern (Goodfellow et al., 2016). Als Anomalie-Score kann der mittlere quadratische Rekonstruktionsfehler verwendet werden:

$$s(x) = MSE(x, \hat{x}) = \frac{1}{d} \sum_{i=1}^d (x_i - \hat{x}_i)^2 \quad (2-2)$$

Dabei bezeichnet d die Zahl der verglichenen Einzelwerte. Für ein Fenster mit H Messzeitpunkten und C Kanälen gilt $d = HC$. Der Score hängt von der Skalierung und der Mittelung ab. Bei standardisierten Eingaben ist er dimensionslos.

Die Annahme lautet, dass ein auf Normaldaten trainierter Autoencoder auffällige Eingaben schlechter rekonstruiert. Das ist keine allgemeine Garantie: Auch bestimmte Anomalien können gut rekonstruiert werden (Ruff et al., 2021). Ob der Score die definierten Versuchsanomalien trennt, muss daher experimentell geprüft werden.

2.3 Messdaten und Fensterbildung

Bei dreiachsigen Messungen enthält jeder Messvektor beispielsweise einen X-, Y- und Z-Wert. Sensorausgaberate, Signalbandbreite und Abfragerate der Software sind unterschiedliche Größen. Häufigeres Auslesen erzeugt nicht automatisch neue Sensorwerte. Die Abstimmung von Sensorkonfiguration und Datenabruf ist daher zu prüfen (Analog Devices, 2022).

Ein Fenster bündelt H aufeinanderfolgende Messvektoren. Bei C Messkanälen enthält es HC Werte. Die Schrittweite S bestimmt, nach wie vielen neuen Messzeitpunkten das nächste Fenster beginnt. Bei gleichmäßiger Erfassung beträgt das Fensterintervall S/f_s . Für die Zeitbewertung wird f_s aus den Zeitstempeln bestimmt; ob jeder erfasste Vektor eine neue Sensormessung enthält, wird getrennt geprüft.

Bei ungleichmäßiger Erfassung und Datenlücken reichen nominelle Zeitangaben nicht aus. Die tatsächlich erfassten Zeitstempel müssen deshalb erhalten bleiben.

2.4 Datenaufteilung und Vorverarbeitung

Trainingsdaten dienen zur Anpassung des Modells. Validierungsdaten unterstützen die Auswahl von Einstellungen und die Kalibrierung. Die Testdaten bleiben bis zur abschließenden Bewertung ungenutzt. Vollständige Aufnahmen bilden dabei Gruppen und

werden vor der Fensterbildung getrennt zugeordnet. So werden ähnliche Abschnitte derselben Aufnahme nicht auf Training und Test verteilt (scikit-learn developers, o. J. b).

Data Leakage liegt vor, wenn Informationen in die Entwicklung einfließen, die bei der vorgesehenen Anwendung nicht verfügbar wären. Ein Beispiel ist die Standardisierung mit Mittelwert und Standardabweichung der gesamten Datenmenge. Solche Parameter dürfen nur aus Trainingsdaten bestimmt werden. Danach werden sie unverändert auf Validierungs-, Test- und Live-Daten angewendet (scikit-learn developers, o. J. a).

Fenster derselben Aufnahme sind keine unabhängigen Messläufe. Auch getrennte Aufnahmen können bei gleicher Montage sehr ähnlich sein. Die Aussagekraft hängt daher von Zahl und Vielfalt der Messläufe ab.

2.5 Schwellenwertkalibrierung

Aus dem Anomalie-Score wird durch einen Schwellenwert eine Entscheidung. Für den Vergleich werden die Scores so ausgerichtet, dass größere Werte eine stärkere Auffälligkeit bedeuten. Die Entscheidung lautet dann anomal, wenn der Score die festgelegte Schwelle überschreitet.

Eine mögliche Kalibrierung nutzt ein hohes empirisches Quantil der Scores separater Normaldaten. Ein Quantil bezeichnet einen Grenzwert in der sortierten Verteilung. Die konkrete Quantilsregel wird für alle Verfahren vor der Testauswertung festgelegt. Damit wird dieselbe Kalibrierungsregel verwendet, auch wenn die numerischen Schwellen verschieden sind.

Eine so bestimmte Schwelle garantiert keine bestimmte Fehlalarmrate auf späteren Aufnahmen. Dafür müssen die neuen Normaldaten ausreichend zur Kalibrierungsgrundlage passen. Eine nachträgliche Anpassung anhand der Testgüte würde die unabhängige Bewertung aufheben (scikit-learn developers, o. J. a).

Eine einzelne Validierungsaufnahme kann zudem nur die darin enthaltenen Normalbedingungen repräsentieren. Zeitlich benachbarte Fenster ersetzen keine unabhängigen Wiederholungen. Wird dieselbe Aufnahme sowohl zur Modellüberwachung als auch zur Schwellenwahl verwendet, sind diese Entwicklungsschritte miteinander gekoppelt; die spätere Testmenge muss dennoch vollständig getrennt bleiben.

2.6 Erkennungsqualität und Aussagekraft der Evaluation

Wenn unabhängige Referenzlabels vorliegen, bezeichnet die positive Klasse die definierte Anomalie. Die Konfusionsmatrix zählt richtig erkannte Anomalien (TP), Fehlalarme (FP), übersehene Anomalien (FN) und richtig erkannte Normalfenster (TN). Daraus ergeben sich die Kennzahlen in Tabelle 2-1 (scikit-learn developers, o. J. c). Die Definitionen erläutern den Qualitätsnachweis, der über den deskriptiven Alarmvergleich dieser Pilotstudie hinausgeht.

Tabelle 2-1: Kennzahlen der binären Anomalieerkennung

Kennzahl	Berechnung	Bedeutung
Precision	$TP / (TP + FP)$	Anteil richtiger Anomalien unter allen Alarmen
Recall	$TP / (TP + FN)$	Anteil erkannter Anomalien unter allen tatsächlichen Anomalien
F1-Score	$2TP / (2TP + FP + FN)$	Harmonisches Mittel von Precision und Recall
Fehlalarmrate (FPR)	$FP / (FP + TN)$	Anteil falsch gemeldeter Fenster unter allen Normalfenstern
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Anteil aller richtigen Entscheidungen

Eine hohe Accuracy kann bei ungleichen Klassenanteilen täuschen. Bei einer gelabelten Evaluation sind deshalb Konfusionsmatrix, Klassenanteile und mehrere Kennzahlen gemeinsam anzugeben. In der vorliegenden Smartphoneuntersuchung fehlen unabhängig erfasste physikalische Ereignisgrenzen. Die Bedienmarker liefern keine hinreichenden Fensterlabels für eine belastbare Konfusionsmatrix. Kapitel 7 berichtet daher Alarmzahlen und relative Alarmzeitpunkte, jedoch keine daraus abgeleiteten F1-Werte oder bestätigten Fehlalarmraten.

2.7 Lokale Ausführung und Zeitverhalten

Bei lokaler Ausführung wird das trainierte Modell nahe der Datenquelle ausgewertet (Shi et al., 2016). Ob Rechenleistung und Arbeitsspeicher der Zielplattform ausreichen, muss für die konkrete Anwendung gemessen werden.

Die Modellgröße bezeichnet die gespeicherte Datei. Der Prozessspeicher umfasst zusätzlich unter anderem Bibliotheken, Eingabedaten und Zwischenergebnisse. Modellgröße und Speicherbedarf während der Ausführung sind daher getrennt auszuweisen. Auch die CPU-Auslastung benötigt eine dokumentierte Messdauer und Bezugsgröße.

Die Inferenzzeit umfasst nur die Modellauswertung. Die Verarbeitungslatenz reicht in dieser Arbeit vom vollständigen Fenster bis zur verfügbaren Entscheidung und enthält auch Vorverarbeitung und Wartezeiten. Die Zeit zum Sammeln des Fensters ist darin nicht enthalten. Ein Echtzeitsystem muss seine Ergebnisse innerhalb festgelegter Fristen liefern (Buttazzo, 2011). Mittelwerte allein belegen das nicht.

P95 und P99 beschreiben hohe empirische Perzentile der gemessenen Latenzen. Sie werden zusammen mit Fristüberschreitungen und Erfassungspausen ausgewertet. Davon zu unterscheiden sind das erste als auffällig markierte Fenster und der Zeitabstand zwischen den ersten Alarmausgaben mehrerer Methoden. Eine Verzögerung ab physischem Störungsbeginn setzt eine unabhängige Zeitreferenz voraus. Trainierte Modelle können für die lokale Ausführung in ein anderes Format konvertiert werden (Google, 2026a). Danach müssen Scores und Entscheidungen erneut geprüft werden.

2.8 Veränderte Normalbedingungen

Eine Änderung der Eingabeverteilung gegenüber den Referenzdaten wird hier als Verteilungsverschiebung bezeichnet. Änderungen der Montage, Sensororientierung oder des Betriebspunkts sind mögliche Ursachen; ein beobachteter Unterschied weist die Ursache allein nicht nach. Ren et al. (2021, Abschnitt IV.B) dokumentieren am Lüfteraufbau nach erneuter Anbringung eine verschobene Rekonstruktionsfehlerverteilung. Solche Veränderungen können die Gültigkeit eines zuvor kalibrierten Normalmodells einschränken.

Für den Smartphoneversuch ist die Bindung an das verwendete Normalprofil eine Gültigkeitsgrenze im Sinne von FF4. Eine systematische Variation normaler Betriebspunkte wird hier nicht ausgewertet. Soll die Pipeline später bei anderer Montage oder anderen Betriebsbedingungen eingesetzt werden, sind unabhängige Normaltests erforderlich. Eine Nachkalibrierung wäre als eigener Versuch zu untersuchen.

3 Stand der Technik und verwandte Arbeiten

Dieses Kapitel vergleicht ausgewählte Originalstudien nach Datenbasis, Lernaufgabe, lokaler Ausführung und Übertragbarkeit. Die gezielte Auswahl mit Recherchestand September 2026 umfasst Arbeiten zu den Forschungsfragen; sie ist keine vollständige systematische Literaturübersicht.

3.1 Datenbasis und Messbedingungen

Lessmeier et al. (2016) untersuchen die überwachte Unterscheidung gesunder Lager sowie von Innen- und Außenringschäden. Sie vergleichen künstlich erzeugte Schäden mit Schäden aus beschleunigten Lebensdauerversuchen. Die Klassifikationsgüte hängt dabei von Schadensherkunft und Verfahren ab. Der Befund betrifft überwachte Lagerklassifikation und lässt sich nicht unmittelbar auf normaldatenbasiertes Training übertragen.

Pittino et al. (2020) untersuchen eine Waferbearbeitungsmaschine bei unterschiedlichen Drehzahlen und Verschleißzuständen. Normaldaten stammen aus funktionsfähigen Kammern, Fehlerdaten aus einem Demonstrator ohne Schmierung. Verglichen werden regressionsbasierte Regelkarten und eine überwachte Support-Vector-Machine-Klassifikation. Letztere nutzt zusätzlich Fehlerbeispiele. Ihr Ergebnis belegt deshalb keinen allgemeinen Vorteil im Normaldaten-Szenario.

Die ausgewertete Messgröße bei Pittino et al. (2020, Abschnitt 2) ist bereits ein aufbereiteter Peak-to-Peak-Vibrationskennwert, ergänzt um die Drehzahl. Die Studie verwendet damit keine unmittelbar vergleichbare dreiachsige Rohdatenrepräsentation.

Knap et al. (2026) vergleichen klassische und tiefe Klassifikationsverfahren in sechs Übertragungsszenarien auf CWRU- und Paderborner Lagerdaten. Ihr beschriebenes Protokoll trennt vollständige Aufnahmen vor der Fensterbildung. Für die eigene Untersuchung folgt daraus die Trennung ganzer Messläufe. Die Übertragbarkeit auf andere Montage- oder Betriebsbedingungen bleibt ein zusätzlicher, hier nicht empirisch abgesicherter Nachweis.

3.2 Verfahren und Edge-Ausführung

Ren et al. (2021) führen mit TinyOL einen Autoencoder auf einem Cortex-M4-Mikrocontroller aus und ermöglichen das lokale Nachlernen seiner letzten Schicht. Dreiachsige Lüftermessungen werden vor der Modelleingabe durch Hauptkomponentenanalyse auf eine Dimension reduziert. Nach erneuter Anbringung des Sensors verändert sich der Rekonstruktionsfehler; die feste Schwelle verursacht mehr Fehlalarme. Die untersuchte Modellanpassung ist von der Bewertung einer unveränderten Pipeline zu unterscheiden.

Antonini et al. (2023) beschreiben ein auf ESP32 und MicroPython beruhendes System für industrielle Einsatzbedingungen mit eingeschränkter Kommunikation. Der Sensorknoten erfasst Daten, trainiert ein Isolation-Forest-Modell lokal und übermittelt bei einer erkannten Auffälligkeit eine Meldung an ein Gateway. Unterwasserpumpen bilden die Zielanwendung.

Die Studie zeigt damit einen Ansatz, bei dem neben der Inferenz auch die Modellanpassung auf dem Sensorknoten stattfindet. Für den Vergleich in dieser Arbeit ist die Trennung von Training und Inferenz wesentlich; aus den Angaben folgt keine Übertragbarkeit der Erkennungsleistung auf den eigenen Lüfteraufbau.

Garay et al. (2026) untersuchen eine multimodale Architektur mit Vibrations-, Akustik- und Thermografiesensorik. Die begutachtete Zeitschriftenfassung verwendet für die Vibrationserkennung zehn Zeitbereichsmerkmale aus 6-s-Fenstern eines Prüfstands mit gezielt erzeugter statischer Unwucht. Verglichen werden PCA, One-Class SVM, Isolation Forest und zwei Zahlenformate eines Autoencoders (Float32 und INT8). Neben einer Referenzplattform wird der Cortex-M4F des Sensorknotens vermessen. Beim Isolation Forest gehören Erkennungswerte zum Offline-Modell mit 100 Bäumen, die MCU-Laufzeit dagegen zu einer verkleinerten Implementierung mit sechs Bäumen. Qualitäts- und Laufzeitangaben müssen daher dem jeweiligen Modellartefakt zugeordnet werden. Andere Eingangsdaten, Fensterlängen und Zielplattformen verhindern eine unmittelbare Übertragung der Rangfolge auf den eigenen Lüfteraufbau.

3.3 Vergleich ausgewählter Arbeiten

Tabelle 3-1 fasst die drei Studien mit eigener Hardwareausführung zusammen. Wegen unterschiedlicher Daten und Messgrenzen erlauben ihre Kennzahlen keine unmittelbare Rangfolge.

Tabelle 3-1: Ausgewählte Vorarbeiten und ihre Bedeutung für die Untersuchung

Arbeit	Daten und Lernaufgabe	Ausführung / Messung	Einordnung für diese Arbeit
Ren et al. (2021)	Lüfter; Autoencoder und Nachlernen	Cortex-M4; Inferenz und Online-Anpassung	Belegt Montageempfindlichkeit; Anpassung getrennt von fester Normalreferenz bewerten
Antonini et al. (2023)	Unterwasserpumpen als Zielanwendung; lokal angepasstes Isolation-Forest-Modell	ESP32 mit MicroPython; lokale Erfassung, Training und Inferenz	Lokale Trainings-/Inferenzpipeline; keine direkte Leistungsrangfolge gegenüber dem eigenen Aufbau
Garay et al. (2026)	Statische Unwucht; zehn Zeitbereichsmerkmale; vier Verfahrensklassen mit zwei Autoencoder-Zahlenformaten	Cortex-M4F; Erkennungswerte und modellbezogener Ausführungsaufwand	Zielplattform, Datenrepräsentation und Modellartefakt zuordnen; keine direkte Rangfolge zum eigenen Aufbau

3.4 Forschungslücke und Einordnung der eigenen Arbeit

Die Vorarbeiten zeigen, dass lokale Anomalieerkennung und die Anpassung an veränderte Messbedingungen bereits untersucht werden. Die eigene Arbeit untersucht an einer

konkreten Messkette, wie sich die Alarmentscheidungen eines neuronalen Verfahrens von zwei Baselines unterscheiden und welchen technischen Aufwand die Ausführung verursacht. Sie liefert eine anwendungsbezogene explorative Vergleichsevaluation; eine allgemeine Forschungslücke oder ein grundsätzlicher Erkennungsvorteil wird daraus nicht abgeleitet.

Dafür werden gleiche Testfenster und eine gemeinsame Kalibrierungsregel für FF2 sowie Messungen am tatsächlich ausgeführten Modell für FF3 verbunden. FF4 ordnet die Nachweisgrenzen ein. Die Smartphoneuntersuchung vergleicht zuerst Einzelprozesse und anschließend dieselben Rohfenster im Replay und im gemeinsamen Live-Prozess. So wird zwischen unterschiedlichen physikalischen Anregungen, identischen Eingaben und zusätzlicher Konkurrenz um Rechenzeit unterschieden.

4 Anforderungsanalyse

Dieses Kapitel formuliert aus den Forschungsfragen, den Grundlagen und den Grenzen der Vorarbeiten überprüfbare Anforderungen an den Smartphonevergleich. Der Katalog dient der nachvollziehbaren Bewertung des im Projektverlauf konkretisierten Versuchs. Er ist kein Nachweis, dass sämtliche Kriterien vor der Datenerhebung registriert wurden. Technische Ziele werden von tatsächlich erbrachten Nachweisen getrennt.

4.1 Messdaten und Vergleichsbasis

Für die Untersuchung muss ein reproduzierbarer Testaufbau realisiert werden (A11). Er soll Messungen bei laufendem Lüfter ohne zusätzliche Anregung sowie mit äußerer Smartphonevibration ermöglichen. Aufbau, Anregung und Zeitreferenz müssen dokumentiert werden (A10). Ein sicherer Betrieb ist Voraussetzung für die Messläufe. Position, Auflage, Rücksetzbarkeit und Ablauf sind so festzuhalten, dass physikalische Wiederholungen überprüfbar werden.

Die Messdaten müssen die benötigten Beschleunigungskanäle mit Einheit, Zeitstempeln und Aufnahmekennung enthalten (A1). Messbereich, nutzbare Bandbreite, Signal-Rausch-Abstand und zeitliche Erfassung sind für den Aufbau zu prüfen. Für eine bestätigende Testserie wären vorab festgelegte Akzeptanzgrenzen nötig. Datenlücken, ungültige Werte und Sättigung sind zu kennzeichnen. Für jede Ereigniskennzeichnung muss erkennbar sein, ob sie eine Bedienhandlung oder einen unabhängig gemessenen physikalischen Zustand beschreibt.

Vollständige Aufnahmen werden vor der Fensterbildung getrennt Training, Validierung oder Test zugeordnet. Fenster dürfen keine Aufnahmegrenzen überschreiten. Training und Schwellenwertkalibrierung verwenden nur Normaldaten. Die Vorverarbeitung wird nach den Regeln aus Abschnitt 2.4 angepasst und unverändert auf spätere Daten angewendet.

Autoencoder und Baselines müssen im direkten Vergleich dieselben Testfenster verwenden (A3). Fensterlänge und Schrittweite werden gemeinsam festgelegt. Unterschiedliche Eingabedarstellungen und der Umgang mit unvollständigen Fenstern sind zu dokumentieren. Die Schwellen folgen derselben Kalibrierungsregel auf separaten Normaldaten. Eingefrorene Artefakte sichern ihre unveränderte Verwendung. Bedienmarker werden für alle Methoden gleich zugeordnet, aber nicht als unabhängig bestätigte Ereignislabels ausgegeben.

4.2 Lokale Verarbeitung und Zeitverhalten

Datenerfassung, Vorverarbeitung und Entscheidung müssen nach dem Start automatisch und ohne Cloud-Dienst auf der Zielhardware laufen (A4). Die Softwareumgebung muss die benötigten Modelloperationen und Schnittstellen unterstützen; Abhängigkeiten und Versionen sind zu dokumentieren (A5, A6). Für jedes gültige Fenster sind Score, Schwelle und die Entscheidung normal/anomal zu speichern. Ungültige Fenster dürfen nicht als

normal bewertet werden. Der Betrieb muss ohne Speicherabbruch und ohne stetig wachsenden Verarbeitungsrückstand möglich sein (A8).

Messläufe müssen gezielt gestartet und geordnet beendet werden können (A13). Jeder Lauf erhält eine eindeutige Kennung. Beim Beenden müssen die bis dahin vollständig erfassten Daten und Ergebnisse gespeichert bleiben. Messdaten, Entscheidungen und die zugehörigen Angaben müssen maschinenlesbar vorliegen und eindeutig zusammengehören (A5).

Als optionale Erweiterung kann eine grafische Benutzeroberfläche zur Live-Anzeige (Live-GUI) entwickelt werden (A12). Sie zeigt den aktuellen Erfassungsstatus sowie Score, Schwelle, Entscheidung und Zeitstempel des zuletzt ausgewerteten Fensters. Fehlende oder veraltete Daten müssen erkennbar sein. Die Anzeige verwendet die Ergebnisse der Verarbeitung und verändert weder Modell noch Schwelle. Der Messbetrieb und die Evaluation müssen auch ohne diese Oberfläche möglich sein.

Für FF3 werden der instrumentierte Auswerteraufwurf und die Verarbeitungslatenz nach Abschnitt 2.7 getrennt betrachtet. Das technische Ziel ist $P99 \text{ der Verarbeitungslatenz} < S/f_s$. Zusätzlich sind Anzahl und Anteil der Überschreitungen gegenüber tatsächlichen Folgefensterankünften sowie Erfassungspausen zwischen Fenstern zu berichten. Messgrenzen und ein gegebenenfalls fehlender isolierter Modellkernzeit-Nachweis müssen erkennbar bleiben.

4.3 Evaluation

Für FF2 werden Alarmzahlen, erste Alarmfenster und relative Ausgabezeitpunkte auf identischen Eingaben ausgewertet (A9). Anzahl und Auswahl der Aufnahmen sowie Ergebnisse je Aufnahme sind anzugeben. Für FF3 werden Modellgröße, CPU-Auslastung und Prozessspeicher auf derselben Zielhardware erfasst (A8). Messbedingungen sowie Software-, Code- und Modellstand müssen nachvollziehbar sein (A5). Bei einer Modellkonvertierung sind numerische Abweichungen und Entscheidungen gegenüber dem Ausgangsmodell anhand dokumentierter Prüfdaten und Akzeptanzgrenzen zu prüfen (A6). Der Einfluss einer Live-GUI wäre separat zu messen.

Für FF4 werden protokollierte Datenintegrität, physikalische Referenz, Aufbaudokumentation und unabhängige Wiederholungen geprüft (A1, A10, A11). Fehlende Nachweise werden ausgewiesen. Die Gleichheit gespeicherter Sensoreingaben belegt die Vergleichbarkeit innerhalb eines Replays, ersetzt aber weder eine Wiederholung der Anregung noch deren unabhängige zeitliche Vermessung.

4.4 Kernanforderungen und Nachweis

Tabelle 4-1 enthält die Anforderungen A1–A13 mit Art, Priorität und Nachweis. F kennzeichnet funktionale Anforderungen an Systemfunktionen. NF bezeichnet nichtfunktionale Anforderungen an Nachvollziehbarkeit, Zeitverhalten und Betriebsstabilität. Methodische Anforderungen (M) betreffen Versuchsplanung und Auswertung. MUSS bezeichnet einen erforderlichen Nachweis für den angestrebten belastbaren Vergleich;

KANN bezeichnet die optionale Live-GUI. Kapitel 6 dokumentiert die Umsetzung, Kapitel 7 bewertet die Erfüllung. Offene MUSS-Nachweise werden nicht durch günstige Einzelergebnisse ersetzt und begrenzen den erreichten Pilotstatus.

Tabelle 4-1: Anforderungen, Prioritäten und Nachweise

ID	Art / Priorität	Anforderung	Nachweis
A1	F MUSS	Messdaten mit überprüfter Qualität erfassen	Kanäle, Einheiten und Zeitstempel prüfen; Bandbreite, Signal-Rausch- Abstand, Sättigung und Datenlücken anhand vorab festgelegter Grenzen bewerten
A2	M MUSS	Datenbasis nach Aufnahmen trennen	Ganze Aufnahmen zuordnen; Training und Kalibrierung nur mit Normaldaten; Vorverarbeitung nur an Trainingsdaten anpassen
A3	M MUSS	Verfahren einheitlich vergleichen	Identische Testfenster im Replay und gemeinsamen Live-Lauf; gemeinsame Kalibrierungsregel und einheitliche Markerzuordnung
A4	F MUSS	Automatisch und lokal auswerten	Versuchslauf ohne Cloud-Dienst; Score, Schwelle und Entscheidung je gültigem Fenster; ungültige Fenster kennzeichnen
A5	NF MUSS	Daten und Softwarestände nachvollziehbar zuordnen	Lauf, Fenster, Code, Modell, Vorverarbeitung sowie Softwareabhängigkeiten und Versionen maschinenlesbar dokumentieren
A6	M MUSS	Zielkompatibilität und ausgeführtes Modell prüfen	Modelloperationen auf der Zielhardware prüfen; bei Konvertierung Scores und Entscheidungen nach vorab festgelegten Akzeptanzgrenzen vergleichen
A7	NF MUSS	Technisches Latenzziel erfüllen: $P99 < S/f_s$	Auswerteraufruf und Verarbeitungslatenz unterscheiden; Latenzverteilung und Überschreitungen tatsächlicher Folgefensterankünfte berichten

ID	Art / Priorität	Anforderung	Nachweis
A8	NF MUSS	Ohne Speicherabbruch und wachsenden Rückstand laufen	Modellgröße, Prozessspeicher, CPU und Rückstand über dokumentierte Messdauer erfassen; Aufwärmphase und Hintergrundlast angeben
A9	M MUSS	Alarmverhalten deskriptiv bewerten	Alarmzahl, erstes Alarmfenster und relative Ausgabezeit auf identischen Eingaben; Ergebnisse je Aufnahme und Grenzen ohne Ereignislabels
A10	M MUSS	Anregung und Ereignisreferenz dokumentieren	Smartphoneposition, Auflage und Vibrationsablauf festhalten; Bedienmarker von unabhängig gemessenem physikalischem Beginn und Ende trennen
A11	M MUSS	Reproduzierbaren Testaufbau realisieren	Aufbau und Protokoll dokumentieren; sicheren Betrieb herstellen; Betrieb ohne und mit zusätzlicher Anregung wiederholen; Rücksetzbarkeit prüfen
A12	F KANN	Live-GUI zur Ergebnisanzeige entwickeln	Angezeigte Werte mit gespeicherten Ergebnissen abgleichen; veraltete und fehlende Daten prüfen; Zusatzaufwand messen
A13	F MUSS	Messläufe starten und geordnet beenden	Start und Stopp prüfen; eindeutige Laufkennung und Erhalt vollständig erfasster Daten und Ergebnisse nachweisen

Kapitel 5 leitet aus A1–A13 Kriterien für Prüfstand, Messkette, Verarbeitung und Evaluation ab. MUSS-Prioritäten sind keine Gewichte: Bekannte Nichterfüllungen dürfen nicht durch Vorteile anderer Kriterien ausgeglichen werden. Noch offene Nachweise begrenzen die Vorauswahl. Die konkrete Lösung wird erst im Variantenvergleich festgelegt.

5 Variantenvergleich und Systementscheidung

Dieses Kapitel begründet die Systemauswahl anhand der Anforderungen A1–A13 und beantwortet damit den zweiten Teil von FF1. Zunächst wird der Prüfstand ausgewählt. Darauf folgen Sensorik und Edge-Plattform, Datenrepräsentation und Verfahren sowie die Entwicklungs- und Inferenzumgebung. Die abschließende Systementscheidung verbindet diese Teilauswahlen mit den noch ausstehenden Nachweisen.

Die folgende Darstellung rekonstruiert die Systementscheidung und die dabei festgelegten Prüfvorbehalte. Die Anforderungen aus Kapitel 4 bilden den Bewertungsrahmen des Smartphonevergleichs. Die Nutzwertmatrizen erläutern Entwurfspräferenzen und sind kein Beleg einer vorab registrierten Auswahlstudie. Ob die praktischen Nachweise im vorhandenen Datenstand erbracht wurden, bewertet Kapitel 7.

5.1 Bewertungsrahmen und Vorgehen

Die Bewertung berücksichtigt die Eignung für die Forschungsfragen, die technische Umsetzbarkeit und den erforderlichen Zusatzaufwand. Anforderungen und Auswahlgründe müssen dabei nachvollziehbar zusammenhängen (NASA, 2023). Für den Prüfstand stehen wiederholbare Zustände und kontrollierte Veränderungen im Vordergrund (A10, A11). Für die Messkette sind Signalqualität und zeitliche Erfassung entscheidend (A1). Hardware und Software müssen die lokale Verarbeitung sowie ihre überprüfbare Ausführung ermöglichen (A4–A8).

Zuerst werden die MUSS-Anforderungen geprüft. Eine bekannte Nichterfüllung kann nicht durch Punkte in anderen Kriterien ausgeglichen werden. Noch offene Nachweise, etwa zur Signalqualität oder Latenz, führen zu einer vorläufigen Auswahl mit Prüfvorbehalt. Sicherheitsmaßnahmen sind vor dem Versuch umzusetzen. Erst unter diesen Voraussetzungen werden geeignete Entwurfsvarianten gegeneinander abgewogen.

Für Prüfstand, Edge-Plattform und Software werden gewichtete Nutzwertmatrizen verwendet. Die Punkteskala reicht von 0 bis 4; ein höherer Wert bedeutet einen höheren Nutzen für dieses Projekt. Die Kriterientabellen erläutern die Bewertungsmaßstäbe. Zwischenwerte 1 und 3 kennzeichnen abgestufte Einschätzungen zwischen den beschriebenen Ankern. Gleich große Punktabstände werden vereinfachend als gleich große Nutzenzuwächse behandelt. Das ist eine Modellannahme und keine gemessene Leistungsrelation.

Die Gewichte beziehen sich auf den Unterschied zwischen dem ungünstigsten und dem günstigsten beschriebenen Zustand eines Kriteriums. Je größer dessen Bedeutung für die Forschungsfragen, desto höher wird dieser Unterschied gewichtet. Dieses Vorgehen orientiert sich am Prinzip der Swing-Gewichtung (Department for Communities and Local Government, 2009, S. 63–65). Die konkreten Gewichte und Punkte sind eigene Entwurfsannahmen; sie stammen weder aus Herstellerangaben noch aus einer

Expertenbefragung. Jede Matrix wird für sich auf 100 % normiert. Nutzwerte verschiedener Auswahlbereiche sind nicht miteinander vergleichbar.

$$N_j = \sum_{i=1}^m \frac{w_i}{100} \cdot s_{ij}$$

Dabei ist m die Anzahl der Kriterien, N_j der Nutzwert der Alternative j, w_i das in Prozent angegebene Gewicht des Kriteriums i und s_{ij} dessen Punktwert. Der Nutzwert liegt zwischen 0 und 4. Er ist weder eine Erfolgswahrscheinlichkeit noch ein Nachweis erfüllter Anforderungen.

Die Sensitivität wird rechnerisch geprüft: Jedes Gewicht wird einzeln um fünf Prozentpunkte erhöht und gesenkt; die übrigen Gewichte werden proportional auf zusammen 100 % angepasst. Zusätzlich werden gleiche Gewichte und ausgewählte Änderungen unsicherer Punktwerte um einen Punkt betrachtet. Diese projektbezogen festgelegten Prüfungen zeigen die Abhängigkeit der Rangfolge von den Annahmen; sie ersetzen keine Messung. Das Vorgehen orientiert sich an der Sensitivitätsanalyse von Nutzwertmodellen (Department for Communities and Local Government, 2009, Abschnitt 6.2.14).

5.2 Bewertung der Prüfstandsalternativen

Der Prüfstand soll einen dokumentierten normalen Betrieb und eine zusätzliche äußere Anregung ermöglichen (A10, A11). Tabelle 5-1 vergleicht drei rotierende Trägersysteme hinsichtlich Kontrollierbarkeit, Zusatzaufwand und Aussagegrenzen. Diese Auswahl betrifft den Träger der Messkette. Die Wahl des Smartphones als externer Anregungsquelle wird anschließend getrennt begründet.

Tabelle 5-1: Bewertung der Prüfstandsalternativen

Alternative	Kontrollierbarkeit und Wiederholung	Zusatzaufwand und Aussagegrenze
Lüfter mit PWM-Ansteuerung	Betriebspunkte vorgebar; Versuchszustände bei fester Montage wiederholbar zu gestalten	Wenige Antriebskomponenten; Lastvariation und Übertragbarkeit auf Industriemaschinen begrenzt
Servomotor mit Regler	Geregelte Drehzahl- und Bewegungsprofile; Lastvariation mit zusätzlicher Belastungseinrichtung	Regler, mechanische Kopplung und gegebenenfalls Lastmaschine nötig; Regelung beeinflusst das Schwingungsverhalten
Motor-/ Lagerprüfstand	Drehzahl, Last und Lagerzustände gezielt untersuchbar	Zusätzliche Mechanik, Messtechnik und Schutzmaßnahmen; größere Nähe zur Lagerdiagnostik

Quelle: Eigene Bewertung auf Grundlage von ARCTIC (o. J.), Oriental Motor (o. J.) und Universität Paderborn (o. J.). Der Aufwand ist eine qualitative Einschätzung der benötigten Komponenten.

Für die Bewertung wird vorausgesetzt, dass bei allen drei Varianten sichere, rücksetzbare Versuchszustände realisiert werden können. Dies bleibt nach A10 und A11 am fertigen

Aufbau nachzuweisen. Die folgenden Kriterien unterscheiden die verbleibenden Gestaltungsmöglichkeiten. Der Realisierungsaufwand erhält das größte Gewicht, weil die Arbeit den Methodenvergleich und keine Entwicklung einer komplexen Antriebsanlage untersucht. Es werden benötigte Baugruppen eingeschätzt; Kosten oder Arbeitsstunden werden nicht behauptet.

Tabelle 5-2: Kriterien und Gewichte für den Prüfstand

Kriterium	Gewicht	Bewertungsmaßstab und Anforderungsbezug
P1 Betriebspunkte einstellen	25 %	A11. 0: keine Vorgabe; 2: einzelne Stellgröße ohne unabhängige Lastregelung; 4: geregelte Drehzahl und zusätzlich Last oder Bewegung vorgebar.
P2 Einflüsse trennen	25 %	A10, A11. 0: Änderungen nicht zuordenbar; 2: mehrere gekoppelte Einflüsse; 4: einfache Variation bei fester Mechanik und wenigen Einflussgrößen.
P3 Aufbauaufwand	40 %	A11. 0: im Projekt nicht realisierbar; 2: Antrieb, Regler und Kopplung aufzubauen; 4: integrierter Antrieb mit wenigen Zusatzbaugruppen.
P4 Anwendungsnähe	10 %	FF1. 0: kein rotierendes System; 2: einfache rotierende Baugruppe; 4: gezielt untersuchbare Maschinen- und Lagerzustände.

Quelle: Eigene Kriterien und Gewichte, abgeleitet aus Kapitel 4. Maßstäbe beschreiben projektbezogene Präferenzen.

Tabelle 5-3: Gewichtete Bewertung der Prüfstandsalternativen

Alternative	P1	P2	P3	P4	Nutzwert
Lüfter mit PWM	2	4	4	2	3,30
Servomotor mit Regler	4	3	2	3	2,85
Motor-/Lagerprüfstand	4	3	1	4	2,55

Quelle: Eigene Punktvergabe auf Grundlage des Variantenvergleichs und der vorausgehenden Kriterien; keine experimentellen Messwerte.

Der Lüfter erhält weniger Punkte bei der Betriebspunktregelung und der Nähe zur Maschinenüberwachung, bietet aber einen einfachen Aufbau mit wenigen zu kontrollierenden Einflussgrößen. Beim Servo kommen Rückführung und mechanische Kopplung hinzu. Der Motor-/Lagerprüfstand ermöglicht die gezielteste Lageruntersuchung, benötigt jedoch zusätzliche Belastungs- und Messbaugruppen. Die Punkte bewerten diese angenommenen Aufbauvarianten; sie belegen keine schlechtere Reproduzierbarkeit eines professionellen Prüfstands.

Unter den gewählten Gewichten wird der Lüfter mit 3,30 Punkten bevorzugt. Beispielhaft ergibt sich $0,25 \cdot 2 + 0,25 \cdot 4 + 0,40 \cdot 4 + 0,10 \cdot 2 = 3,30$. In den acht einzelnen

Gewichtsänderungen bleibt er vorn. Bei gleichen Gewichten erreichen dagegen alle Varianten 3,00 Punkte. Die Auswahl hängt damit wesentlich vom begrenzten Realisierungsaufwand ab. Eine einzelne Änderung eines Punktwerts um einen Punkt kehrt die Ausgangsrangfolge an der Spitze nicht um; gemeinsame Änderungen können dies tun.

Für den Kernversuch wird deshalb der Lüfter als Träger der Messkette gewählt. Ein Servomotor wäre bei Lastwechseln oder dynamischen Bewegungsprofilen sinnvoll, ein Motor-/Lagerprüfstand bei konkreten Lagerfehlern. Die zusätzliche Anregung erfolgt hier durch die Vibrationsfunktion eines Smartphones auf dem Tisch. Sie ermöglicht einen praktisch umsetzbaren Pilottest des Alarmverhaltens. Eine kalibrierte Anregungsstärke oder reproduzierbare Übertragungsfunktion folgt daraus nicht; ein Vergleich alternativer Vibrationsquellen wurde nicht gemessen.

Als konkrete Variante wird der ARCTIC P12 Pro PST verwendet, dessen Datenblatt PWM-Ansteuerung und ein Drehzahlsignal ausweist (ARCTIC, o. J.). Ein PWM-Tastgrad ist eine Stellgröße und kein Nachweis einer konstanten Drehzahl. Die tatsächliche Drehzahl wurde nicht erfasst. Der untersuchte Betrieb wird deshalb ausschließlich durch die dokumentierte PWM-Vorgabe von 75 % beschrieben. Kapitel 6 erläutert den Smartphoneaufbau und den Messablauf, Kapitel 7 die Befunde und Grenzen.

5.3 Auswahl von Sensorik und Edge-Plattform

Die Sensorik muss die vorgesehenen dreiachsigen Beschleunigungssignale mit ausreichender Qualität erfassen. Verglichen werden zwei digital anschließbare MEMS-Sensoren und eine industrielle Messkette. Tabelle 5-4 nennt Eigenschaften, die für A1 relevant sind. Datenblattwerte belegen noch keine ausreichende Signalqualität am späteren Montageort.

Tabelle 5-4: Vergleich der Sensorik und Messketten

Alternative	Relevante Herstellerangaben	Bewertung für den Prüfstand
ADXL345	Drei Achsen; $\pm 2/4/8/16$ g; I ² C/SPI; FIFO	Direkte digitale Erfassung; Bandbreite, Auflösung und Rauschen am Aufbau prüfen
ADXL355	Drei Achsen; $\pm 2/4/8$ g; 20-Bit-ADC; typische Rauschdichte $22,5 \mu\text{g}/\sqrt{\text{Hz}}$ bei ± 2 g	Alternative bei höheren Anforderungen an Auflösung und Rauschen; eigene Integrationsprüfung nötig
PCB 356A32 mit IEPE-Messkette	Drei Achsen; ± 50 g; 100 mV/g; 1–4000 Hz innerhalb ± 5 %	Größerer spezifizierter Frequenzbereich; Konstantstromspeisung und externe A/D-Wandlung nötig

Quelle: Eigene Einordnung nach Analog Devices (2022, 2025) und PCB Piezotronics (o. J.). Die Zahlen sind Herstellerangaben.

Für den ersten Systementwurf wird der ADXL345 mit digitaler I²C-Anbindung vorgesehen. Er bietet eine vollständige dreiachsige Messung ohne externe A/D-Wandlung. Auch der ADXL355 erfüllt dieses Integrationsprinzip; die Wahl des ADXL345 ist daher eine zu prüfende

Ausgangsentscheidung und kein Nachweis seiner Überlegenheit. Reichen Auflösung oder Signal-Rausch-Abstand nicht aus, wird der ADXL355 als Alternative geprüft. Eine IEPE-Messkette ist erst bei entsprechendem Bandbreiten- oder Messqualitätsbedarf zu rechtfertigen.

Zu prüfen sind effektive Erfassungsrate, neue Sensorwerte, Datenlücken, Sättigung und der Abstand des Nutzsignals zum Rauschboden bei dokumentierter Montage (A1). Die Akzeptanzgrenzen sind vor der abschließenden Testserie festzulegen. Beim ADXL345 beträgt die maximal empfohlene Sensorausgaberate bei 400-kHz-I²C 800 Hz; bei 100-kHz-I²C sind es 200 Hz. Diese Grenzen ersetzen keine Messung der tatsächlich erreichten Erfassung (Analog Devices, 2022, S. 17).

Die Edge-Plattform muss neben der Inferenz auch Datenerfassung, Vorverarbeitung, Baselines und Messwerkzeuge tragen (A4, A7, A8). Tabelle 5-5 vergleicht deshalb den Ressourcenrahmen und die daraus abgeleiteten Integrationsfolgen. Taktraten und Beschleunigerangaben erlauben keinen unmittelbaren Vergleich der Inferenzzeiten.

Tabelle 5-5: Bewertung der Edge-Plattformen

Alternative	Dokumentierter Ressourcenrahmen	Bewertung für den Methodenvergleich
STM32H743	Cortex-M7 bis 480 MHz; bis 2 MB Flash und 1 MB RAM	Hardwarenahe Umsetzung möglich; knapper Speicher und eigenständige Portierung der Pipeline
Raspberry Pi 4 Model B	Vier Cortex-A72-Kerne mit 1,8 GHz; RAM im GB-Bereich	Plausible Linux-Plattform für CPU-Inferenz und Messwerkzeuge
Raspberry Pi 5	Vier Cortex-A76-Kerne mit 2,4 GHz; RAM im GB-Bereich	Nominell größere CPU-Reserve; konkrete Latenz und Speicherbelegung bleiben zu messen
Jetson Orin Nano Super	Sechs Cortex-A78AE-Kerne; 8 GB RAM; GPU-Beschleunigung	Sinnvoll bei GPU-bedürftigen Modellen; Nutzen für den vorgesehenen CPU-Vergleich nicht belegt

Quelle: Eigene Bewertung nach STMicroelectronics (2026), Raspberry Pi Ltd. (2026a, 2026b) und NVIDIA (o. J.).

Die Ressourcenangaben werden nicht in erwartete Inferenzzeiten umgerechnet. Die gewichtete Vorauswahl verbindet den nominellen Ressourcenrahmen mit der Integration der vollständigen Pipeline. Das Gewicht für zeitliche Kontrollierbarkeit bleibt begrenzt, weil der Bewertungsmaßstab zu FF3 ein empirisches Latenzziel und keine harte Echtzeitgarantie formuliert.

Tabelle 5-6: Kriterien und Gewichte für die Edge-Plattform

Kriterium	Gewicht	Bewertungsmaßstab und Anforderungsbezug
E1 Nomineller Ressourcenrahmen	35 %	A8. 0: keine plausible Reserve; 2: eingeschränkter Rahmen; 4: GB-Arbeitsspeicher und größere nominelle CPU-Reserve im Variantenfeld. Kein Laufzeitnachweis.

Kriterium	Gewicht	Bewertungsmaßstab und Anforderungsbezug
E2 Integration der Gesamtpipeline	35 %	A4–A6. 0: keine geeignete Umgebung; 2: erhebliche Anpassungen; 4: allgemeine Linux-/Python-Umgebung für Erfassung, Verfahren und Messwerkzeuge.
E3 Zeitliche Kontrollierbarkeit	15 %	A7. 0: Ablauf kaum beeinflussbar; 2: allgemeines Betriebssystem mit schwankender Last; 4: hardwarenah planbare Ausführung. Keine Fristgarantie.
E4 Passung zum CPU-Kernvergleich	15 %	FF1, FF3. 0: ungeeignete Plattformklasse; 2: zusätzliche Beschleuniger bleiben ungenutzt oder die Plattform erzwingt einen Portierungsfokus; 4: CPU-System ohne solche Zusatzschwerpunkte.

Quelle: Eigene Kriterien und Gewichte aus Kapitel 4. Ressourcen, Softwareintegration, Zeitplanung und Untersuchungsumfang werden getrennt betrachtet.

Tabelle 5-7: Gewichtete Bewertung der Edge-Plattformen

Alternative	E1	E2	E3	E4	Nutzwert
STM32H743	1	1	4	2	1,60
Raspberry Pi 4	3	4	2	4	3,35
Raspberry Pi 5	4	4	2	4	3,70
Jetson Orin Nano Super	4	4	2	2	3,40

Quelle: Eigene Punktvergabe nach dem dokumentierten Ressourcenrahmen und den Kriterien E1–E4; keine gemessenen Leistungswerte.

Der STM32 wird bei Ressourcen und Integration wegen des begrenzten Speichers und der notwendigen Portierung niedrig bewertet. Die Linux-Plattformen erhalten bei der allgemeinen Integration denselben Wert. Der Pi 4 erhält für E1 einen Punkt weniger als der Pi 5; dies bewertet ausschließlich die nominelle CPU-Reserve. Die Jetson-GPU bringt für den hier abgegrenzten CPU-Vergleich keinen zusätzlichen Untersuchungsnutzen. Ihre Leistungsfähigkeit wird damit nicht infrage gestellt.

Der Raspberry Pi 5 erreicht 3,70 Punkte und wird als Zielplattform gewählt. Er bleibt bei den acht einzelnen Gewichtsänderungen und bei gleichen Gewichten vorn. Die Einschätzung ist jedoch punktempfindlich: Wird seine Ressourcenbewertung E1 um einen Punkt gesenkt, liegt er mit 3,35 gleichauf mit dem Pi 4 und unter dem Jetson mit 3,40. Die nominelle Reserve ist deshalb eine Entwurfsbegründung, kein Leistungsbeleg. Die Erfüllung von A7 und A8 wird an der vollständigen Pipeline gemessen; ein zusätzlicher Hardwarebenchmark gehört nicht zum Kernversuch.

5.4 Datenbasis, Signalrepräsentation und Verfahren

Für die Evaluation werden eigene reale Messläufe gewählt. Damit umfasst die Untersuchung die vollständige Kette vom Sensor bis zur Entscheidung. Öffentliche Datensätze können die Methodenentwicklung unterstützen, ersetzen jedoch keinen Nachweis an der gewählten Messkette. Simulierte Daten eignen sich für Funktionstests, belegen aber keine Erkennungsleistung am realen Prüfstand. Ganze Aufnahmen werden nach A2 getrennt zugeordnet. Training und Schwellenwertkalibrierung verwenden ausschließlich Normaldaten.

Tabelle 5-8: Bewertung der Signalrepräsentationen

Darstellung	Informationsgehalt und Grenze	Vorgesehene Rolle
Dreiachsiges Zeitfenster	Erhält Reihenfolge und Achsenbeziehungen; höhere Eingabedimension	Gemeinsame Ausgangsdaten für den Hauptvergleich
Zeitbereichs-kennwerte	Kompakt und interpretierbar; zeitliche Struktur wird verdichtet	RMS als statistische Baseline
Frequenzspektrum mittels FFT	Zeigt Frequenzanteile; abhängig von Abtastung, Fensterdauer und Fensterfunktion	Mögliche ergänzende Analyse; kein ausgewerteter Spektralnachweis des Smartphoneversuchs
Zeit-Frequenz-Darstellung	Zeigt zeitlich wechselnde Frequenzanteile; zusätzliche Parameter und Rechenaufwand	Keine zusätzliche Variante im Kernvergleich

Quelle: Eigene Einordnung auf Grundlage von Ruff et al. (2021) und SciPy developers (o. J. a, o. J. b).

Die Fast Fourier Transform (FFT) ist ein effizienter Algorithmus zur Berechnung der diskreten Fourier-Transformation. Sie kann ergänzend eingesetzt werden, um beobachtbare Frequenzanteile zu untersuchen. Für H gleichmäßig abgetastete Werte beträgt der Abstand der Frequenzstützstellen $\Delta f = f_s/H$; dies allein beschreibt noch nicht die Trennbarkeit benachbarter Frequenzanteile (SciPy developers, o. J. a, o. J. b). Ein experimenteller Spektralnachweis der Smartphoneanregung ist nicht Teil der hier ausgewerteten Ergebnisse.

Eine FFT behebt weder Datenlücken noch Aliasing. Vor einer ergänzenden Spektralauswertung müssen zeitliche Regelmäßigkeit, nutzbare Sensorbandbreite und mögliche Frequenzüberlagerungen durch die Abtastung geprüft werden. Mittelwertbehandlung, Fensterfunktion und Amplitudenskalierung wären zu dokumentieren. FFT-Merkmale würden eine weitere Verfahrensvariante bilden und müssten vor ihrer Testauswertung festgelegt sowie mit ihrem Zusatzaufwand bewertet werden. Die FFT selbst liefert keine Anomalieentscheidung.

Tabelle 5-9: Auswahl der Anomalieverfahren

Verfahren	Rolle und Auswahlgrund	Wesentliche Grenze
Autoencoder	Neurales Hauptverfahren; prüft den Nutzen einer erlernten multivariaten Rekonstruktion	Eine gute Trennung der Zustände ist empirisch zu prüfen
RMS mit Schwelle	Statistische Minimalbaseline; transparente Verdichtung auf einen Kennwert	Verwirft die zeitliche Reihenfolge
Isolation Forest	Klassische Lernbaseline; ergänzt den RMS um eine multivariate Bewertung	Nutzt keine explizite zeitliche Nachbarschaft
One-Class SVM	Plausible Alternative zur klassischen Lernbaseline	Zusätzliche Kernel- und Hyperparameterwahl; keine weitere Vergleichsrolle
Überwachte Klassifikation	Geeignet bei repräsentativen gelabelten Fehlerklassen	Benötigt Fehlerbeispiele im Training und passt damit nicht zum gewählten Szenario

Quelle: Eigene Rollenverteilung nach Liu et al. (2008), Ruff et al. (2021) und scikit-learn developers (o. J. d).

Der Hauptvergleich umfasst Autoencoder, RMS und Isolation Forest. So werden die in Kapitel 1 festgelegten drei Verfahrensklassen abgedeckt. Alle erhalten dieselben $H \times 3$ -Ausgangsfenster. Die Standardisierung wird ausschließlich an Trainingsdaten angepasst. Der Autoencoder rekonstruiert das standardisierte Fenster, der Isolation Forest verwendet dessen Werte in fester Reihenfolge als Vektor. Die RMS-Baseline berechnet einen gemeinsamen Effektivwert über alle $3H$ standardisierten Werte; dieser Score ist dimensionslos und vom RMS der Beschleunigung in g zu unterscheiden.

Für jedes Verfahren wird die Schwelle als empirisches 99. Perzentil seiner Scores auf einer separaten normalen Validierungsaufnahme bestimmt. Größere Scores bedeuten stets eine stärkere Auffälligkeit. Die genaue Quantilsberechnung, Modellparameter und Fensterbildung sind im eingefrorenen Profil dokumentiert (A2, A3). Der Vergleich untersucht das beobachtete Alarmverhalten; ein grundsätzlicher Erkennungsvorteil des Autoencoders wird nicht vorausgesetzt.

5.5 Kriterienbasierte Bewertung der Softwarealternativen

Die Softwareauswahl betrifft den Entwicklungs- und Ausführungspfad des Autoencoders. TensorFlow/Keras und PyTorch dienen zur Modellbildung; TFLite/LiteRT, ExecuTorch und ONNX Runtime bieten Wege zur lokalen Inferenz. Der Prüfstand wurde bereits in Abschnitt 5.2 bewertet. Die nachfolgenden Kriterien betreffen die Software und werden aus A4–A8 abgeleitet.

Tabelle 5-10: Kriterienbasierter Vergleich der Softwarealternativen

Entwicklungs- und Ausführungspfad	Übergang zur Inferenz	Lokale Ausführung	Integrations- und Prüfaufwand
TensorFlow/ Keras → TFLite/LiteRT	Dokumentierter Konverter für Keras-Modelle; .tflite-Datei	CPU-Inferenz mit passender Laufzeit	Modelloperationen, Paketkompatibilität und Konvertierung prüfen
PyTorch → ExecuTorch	Modellexport und Anpassung an ein Backend; .pte-Datei	Arm-CPU über geeignetes Backend, z. B. XNNPACK	Export, Backend und numerische Übereinstimmung prüfen
PyTorch → ONNX Runtime	Export in das ONNX-Austauschformat	CPU-Paket für Linux ARM64	Exporter, ONNX-Operatorversionen und Laufzeit abstimmen
TensorFlow/ Keras → TensorFlow direkt	Kein Wechsel des Modellformats erforderlich	TensorFlow-CPU-Paket für Linux ARM64	Paketkompatibilität und Ressourcenbedarf der vollständigen Umgebung prüfen

Quelle: Eigene Bewertung nach Google (2026a, 2026c, 2026d), PyTorch (o. J.) und ONNX Runtime developers (o. J.). Die Zielkompatibilität des konkreten Modells ist bei jeder Alternative zu prüfen.

Die gemeinsame Voraussetzung ist, dass Modelloperationen, Python-Version und Betriebssystem auf dem Zielgerät zusammenpassen (A6). Dies ist für das konkrete Modell noch nachzuweisen. Dokumentierte ARM64-Unterstützung allein bestätigt keine beliebige Raspberry-Pi-OS-Konfiguration. Für TFLite wird ein separates Interpreter-Paket beschrieben; Modelle mit zusätzlichen TensorFlow-Operationen können eine umfangreichere Umgebung benötigen (Google, 2026c). Diese Einschränkung wird bei der Modellkonvertierung geprüft.

Tabelle 5-11: Kriterien und Gewichte für die Softwareauswahl

Kriterium	Gewicht	Bewertungsmaßstab und Anforderungsbezug
S1 Entwicklung und Inferenz trennen	30 %	A5, A8. 0: kein getrennt ladbares Modell; 2: gespeichertes Modell benötigt vollständiges Framework; 4: separates Inferenzpaket vorgesehen.
S2 Modellübergang nachvollziehen	25 %	A5, A6. 0: kein dokumentierter Weg; 2: nur teilweise dokumentiert; 4: offizieller Weg zum Speichern bzw. Exportieren und Laden.
S3 Python-Anbindung	25 %	A4, A5. 0: keine Anbindung; 2: eigene Brücke nötig; 4: offiziell dokumentierte Python-API.

Kriterium	Gewicht	Bewertungsmaßstab und Anforderungsbezug
S4 Linux-ARM64-CPU unterstützen	20 %	A4, A6. 0: kein dokumentierter Weg; 2: eigene Portierung nötig; 4: dokumentiertes CPU-Paket für Linux ARM64. Konkrete Versionen bleiben zu prüfen.

Quelle: Eigene Kriterien und Gewichte aus Kapitel 4. Die Punkte bewerten dokumentierte Wege, nicht deren gemessenen Ressourcenbedarf.

Tabelle 5-12: Gewichtete Bewertung der Softwarealternativen

Entwicklungs- und Ausführungspfad	S1	S2	S3	S4	Nutzwert
TensorFlow/Keras → TFLite/LiteRT	4	4	4	4	4,00
PyTorch → ExecuTorch	4	4	4	4	4,00
PyTorch → ONNX Runtime	4	4	4	4	4,00
TensorFlow/Keras → TensorFlow direkt	2	4	4	4	3,40

Quelle: Eigene Bewertung der offiziellen Dokumentation von Google (2026a, 2026c, 2026d), PyTorch (o. J.) und ONNX Runtime developers (o. J.).

Die drei separaten Inferenzwege erreichen jeweils 4,00 Punkte. Sie bieten die in S1–S4 beschriebenen Möglichkeiten. Die direkte TensorFlow-Ausführung erhält bei S1 zwei Punkte, weil dabei das vollständige Framework vorgesehen ist; dafür entfällt der Formatwechsel. Eine hohe Punktzahl sagt nichts über spätere Latenz, Speichernutzung oder Erkennungsqualität aus. Diese Größen werden nicht mit unbelegten Schätzwerten bewertet.

Der Gleichstand bleibt bei gleichen Gewichten und bei den acht einzelnen Gewichtsänderungen bestehen. Sinkt ein bisher mit vier Punkten bewertetes Kriterium einer Alternative um einen Punkt, verliert diese ihren Gleichstand an der Spitze. Die Matrix identifiziert mehrere nach den gewählten Kriterien gleich bewertete Wege. Die anschließende Festlegung auf TensorFlow/Keras mit TFLite ist eine zusätzliche, offen gelegte Entwurfsentscheidung; sie folgt nicht aus einem eindeutigen Nutzwertvorsprung.

Für diese Arbeit wird aus dieser Gruppe TensorFlow/Keras mit einem Float32-TFLite-Modell gewählt. Der dokumentierte direkte Konverter für Keras-Modelle ermöglicht einen klaren Weg von der Modellbildung zur gespeicherten Inferenzdatei und deren Prüfung gegen das Ausgangsmodell (Google, 2026a). Damit wird eine konkrete Umsetzung innerhalb der gleich bewerteten Gruppe festgelegt. ExecuTorch bleibt eine gleichwertige Entwurfsalternative nach den verwendeten Maßstäben; ONNX Runtime wäre besonders bei einem gewünschten Austauschformat zweckmäßig. Ein solcher Austausch ist keine eigene Anforderung des Kernversuchs.

Im Hauptvergleich wird auf Quantisierung verzichtet. So wird zunächst keine zusätzliche Änderung der Zahlendarstellung untersucht. Die Dokumentation empfiehlt ein nicht quantisiertes TFLite-Modell als Ausgangspunkt für die Kompatibilitätsprüfung (Google, 2026b). Ob die ausgewählte Laufzeit A7 und A8 erfüllt, entscheidet erst die Messung. Auch eine kleine Modelldatei garantiert keinen kleinen Prozessspeicher. Konkrete Pakete, Versionen und ihre Einbindung werden in Kapitel 6 dokumentiert.

5.6 Systementscheidung und Übergang zur Implementierung

Die abschließende Tabelle fasst die Entscheidungen und die jeweils noch erforderlichen Nachweise zusammen. Die Nutzwerte unterstützen die Auswahl innerhalb des jeweiligen Variantenfelds. Sie sind an die offengelegten Gewichte, Punkte und Prüfvorbehalte gebunden. Die praktische Eignung bleibt Gegenstand von Implementierung und Evaluation.

Tabelle 5-13: Systementscheidungen und ausstehende Nachweise

Bereich	Auswahl	Anforderungsbezug und Nachweis
Prüfstand	Lüfter mit PWM; 3,30 Punkte	A10, A11: dokumentierte Smartphoneanregung und reproduzierbarer Aufbau
Sensorik	ADXL345 als Ausgangsvariante; ADXL355 als Alternative	A1: Signalqualität und zeitliche Erfassung vor der Testserie prüfen
Edge-Plattform	Raspberry Pi 5; 3,70 Punkte	A4, A7, A8: vollständige Pipeline und Zeitbudget messen
Datenbasis	Eigene reale Messläufe; Normaldaten für Training und Kalibrierung	A2, A3, A9: getrennte Aufnahmen und gemeinsame Testbasis
Verfahren	Autoencoder, RMS und Isolation Forest	A3, A9: gleiche Fenster und Kalibrierungsregel; Alarmverhalten vergleichen
FFT	Mögliche ergänzende Spektralanalyse; kein Ergebnisgegenstand	A1: nutzbare Messbandbreite vor einer zusätzlichen Frequenzbewertung prüfen
Neuronale Software	TensorFlow/Keras → Float32-TFLite; 4,00 Punkte, Gleichstand mit zwei Alternativen	A5–A8: Konvertierung, Zielkompatibilität und Ressourcen prüfen
Bedienung und Ausgabe	Start/Stopp und gespeicherte Ergebnisse; Live-GUI optional	A4, A5, A12, A13: Datenzuordnung, geordnetes Beenden und gegebenenfalls Anzeige prüfen

Quelle: Eigene Zusammenführung der Entscheidungen aus den Abschnitten 5.2–5.5.

6 Implementierung und Versuchsdesign

6.1 Smartphoneversuch und Untersuchungsziele

Die explorative Untersuchung vom 19. September 2026 verwendet eine zusätzliche mechanische Anregung durch die Vibrationsfunktion eines angerufenen Smartphones. Das Smartphone liegt auf dem Tisch; der vorgesehene Übertragungsweg führt über Tisch und Lüfteraufbau zum Beschleunigungssensor. Der Lüfter läuft mit einer PWM-Vorgabe von 75 %. Die Smartphonevibration ist eine äußere Einwirkung und kein diagnostisch bestätigter Lüfter- oder Lagerschaden.

Bewertet werden Alarmentscheidungen, relative Ausgabezeitpunkte und technische Ausführung auf dem Raspberry Pi. Bedienhandlungen sind protokolliert, der physikalische Beginn und das Ende der Vibration jedoch nicht unabhängig gemessen. Die Untersuchung liefert deshalb deskriptive Befunde zum Alarmverhalten und keine bestätigende Prüfung der Erkennungsqualität anhand vollständiger Ereignislabels.

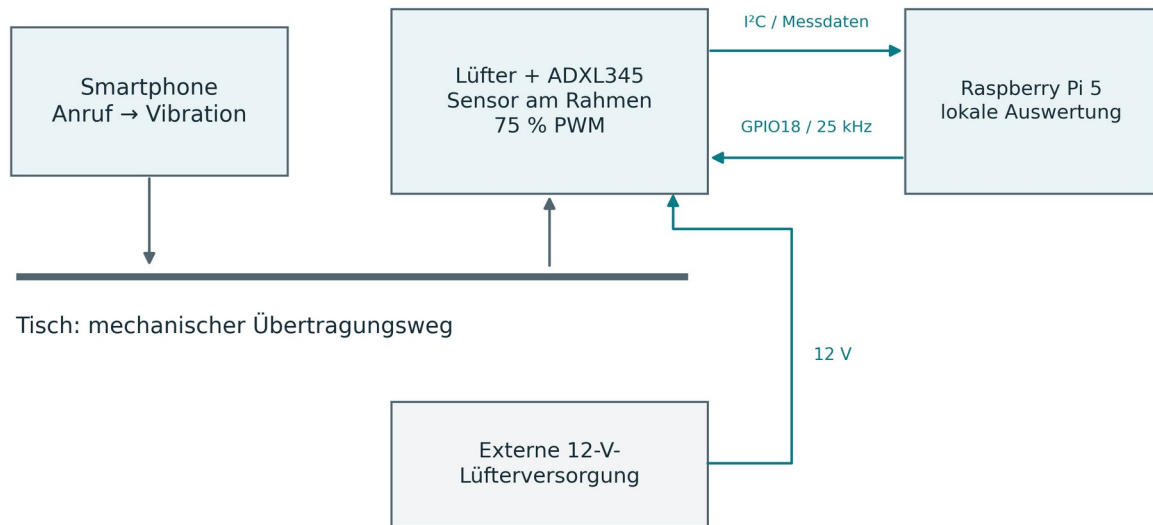
Alle Smartphoneaufnahmen verwenden das Profil `standlauf_pwm75_200hz` vom 18. September. Trainingsaufnahmen, Modellgewichte, Skalierungsparameter und Schwellen sind vor der Versuchsauswertung festgelegt und als Bündel erhalten. Die Artefakte bleiben in Einzelbetrieb, Replay und gemeinsamem Live-Lauf unverändert. Daten- und Quellcodebelege sind im Anhang als B1–B4 zugeordnet.

Die Untersuchung besitzt drei Ebenen: Einzelprozesse zeigen die jeweilige Methode ohne die beiden anderen Auswerter; ein Offline-Replay vergleicht alle Methoden auf denselben gespeicherten Rohfenstern; der gemeinsame Live-Lauf vergleicht zusätzlich die tatsächlichen Ausgabezeitpunkte auf einer gemeinsamen Aufnahme. Nur der letzte Modus verbindet identische aktuelle Sensoreingaben mit gleichzeitig aktiven Auswertern.

6.2 Hardware und mechanischer Übertragungsweg

Der Aufbau besteht aus einem Raspberry Pi 5, einem über I²C angeschlossenen ADXL345 und einem ARCTIC P12 Pro PST mit externer 12-V-Versorgung. Der Sensor ist am feststehenden Lüfterrahmen befestigt. Das Profilprotokoll enthält eine Bestätigung der festen Montage durch den Versuchsverantwortlichen. Eine metrische Vermessung der Orientierung oder der mechanischen Übertragungsfunktion liegt nicht vor.

Das Smartphone regt den Tisch an; die über den Aufbau übertragene Bewegung wird am ADXL345 erfasst. Ein Vergleich getrennter Anrufe setzt kontrollierte Smartphoneposition, Auflage, Vibrationsmuster und Umgebungsbedingungen voraus. Diese Bedingungen sind nicht unabhängig vermessen; auch das Planungsprotokoll enthält keine vollständig bestätigte Gerätebeschreibung oder Geometrie. Die Untersuchung bleibt deshalb eine Pilotstudie. Im gemeinsamen Live-Lauf ist die Gleichheit der Eingaben durch den gemeinsamen Rohdatenstrom und die Fensterhashes belegt.



Schema, nicht maßstäblich. Position und Anregungsstärke sind nicht vermessen.

Abbildung 6-1: Schematischer Aufbau des Smartphoneversuchs. Die Darstellung erklärt den vorgesehenen Übertragungsweg; sie ist kein Foto und bestätigt keine nicht vermessene Geometrie. Quelle: eigene Darstellung anhand von B1.

Tabelle 6-1: Hardware und Betriebszustand der Smartphoneuntersuchung

Bestandteil	Dokumentierter Stand	Grenze des Nachweises
Zielrechner	Raspberry Pi 5, ARM64, vier logische Kerne	Keine Aussage über andere Rechner
Lüfter	ARCTIC P12 Pro PST, externe 12 V	Keine kontinuierliche Spannungs- oder Drehzahlmessung
Sensor	ADXL345 am feststehenden Lüfterrahmen	Orientierung und Übertragungsfunktion nicht kalibriert
Stellanschluss	BCM-GPIO18, physischer Pin 12; Hardware-PWM	Rücklesung der Einstellung, keine elektrische Signalprüfung
Betriebspunkt	75 % Tastgrad bei 25 kHz	Keine Gleichsetzung mit 75 % Drehzahl
Anregung	Smartphonevibration auf dem Tisch durch Anruf	Beginn, Ende und Amplitude nicht unabhängig gemessen
Vergleichseinheit	128 aufeinanderfolgende XYZ-Punkte	Fenster sind innerhalb eines Ereignisses zeitlich abhängig

6.3 PWM, Sensor und Zeitbasis

Die protokollierte Hardware-PWM besitzt eine Periode von 40.000 ns, eine Einschaltdauer von 30.000 ns und enable = 1; daraus folgen 25 kHz und 75 % Tastgrad. Diese Einstellung wird vor den Aufnahmen zurückgelesen. Die Anomalieentscheidung steuert den Lüfter nicht. Das Beenden des Smartphone-Recorders löst keinen automatischen 0%-Befehl aus; Aufnahmeende und Beendigung des Lüfterbetriebs sind getrennte Vorgänge.

Der ADXL345 arbeitet nominell mit 200 Hz, ± 2 g, voller Auflösung und FIFO-Stream-Erfassung. Der Umrechnungsfaktor beträgt 0,0039 g pro LSB. I²C-Bus 1 ist mit 100 kHz konfiguriert. Registerrücklesungen und Qualitätsmerkmale sind in run.json und den Rohjournalen enthalten. Ein CSV-Datensatz enthält einen XYZ-Messpunkt, keine drei getrennten Zeitpunkte.

Die nominelle –3-dB-Sensorbandbreite beträgt gemäß Datenblatt die Hälfte der eingestellten Ausgaberate, hier 100 Hz. Dies bestätigt weder aliasfreie Messungen noch einen ausreichenden Signal-Rausch-Abstand am Montageort. FIFO-Stream-Erfassung unterstützt die Zuordnung neuer Messwerte; die protokollierte Hostrate ersetzt keine unabhängige Kalibrierung des Sensortakts (Analog Devices, 2022, S. 4, 21 und 32).

Die monotone Host-Uhr dokumentiert den Abschluss der Messpunktlesung, der Fensterbildung und der Entscheidung. UTC dient der Zuordnung von Bedienereignissen. Nominell dauert ein nicht überlappendes Fenster $128/200 = 0,640$ s. Aus den Host-Zeitstempeln ergeben sich in den ausgewählten Aufnahmen rund 207 XYZ-Punkte pro Sekunde beziehungsweise etwa 0,617 s pro Fenster. Diese Beobachtung ist keine unabhängige Messung des internen Sensortakts; die Abweichung bleibt offengelegt. Die Fensterfüllzeit gehört zur praktischen Alarmverzögerung, nicht zur nachfolgend gemessenen reinen Verarbeitung eines bereits vollständigen Fensters.

Bei erkannten Datenlücken, Überläufen oder Sättigung wird ein Fenster nicht stillschweigend als normal gewertet. Unvollständige Reste werden gespeichert, aber nicht durch künstliche Werte zu vollständigen Fenstern ergänzt. Die begrenzte Queue umfasst vier Fenster. Im gemeinsamen Lauf wurde höchstens ein wartendes Fenster protokolliert.

6.4 Vorverarbeitung und eingefrorene Modelle

Die Pipeline verarbeitet die Achsen X, Y und Z in fester Reihenfolge als Float32. Sie verwendet die auf normalen Trainingsdaten bestimmten Mittelwerte μ und Standardabweichungen σ : $z = (x - \mu) / \sigma$. Der gleiche Scaler wird für alle drei Methoden eingesetzt. Vor dieser Standardisierung wird kein eigener Mittelwert je Fenster entfernt. Damit können auch Änderungen des Fenstermittelwerts gegenüber der Trainingsreferenz in die Scores eingehen. Diese Festlegung gilt unverändert für Training, Kalibrierung, Replay und Live-Auswertung.

Tabelle 6-2: Festgelegte Skalierung des aktuellen Normalprofils, gerundet

Achse	Trainingsmittelwert μ [g]	Trainingsstandardabweichung σ [g]
X	–0,624318715	0,028777793
Y	–0,710917289	0,024151856
Z	0,096116970	0,080470853

Der RMS-Score ist die Quadratwurzel des Mittelwerts aller 384 quadrierten standardisierten Werte eines Fensters. Er ist ein dimensionsloser aggregierter Kennwert und nicht der

unskalierte Beschleunigungs-RMS in g. Isolation Forest verarbeitet dieselben 384 Werte als abgeflachten Merkmalsvektor. Verwendet werden 200 Bäume, `random_state = 42`, `max_samples = auto`, `contamination = auto` und `n_jobs = 1`. Der Anomaliescore ist `-score_samples`; die Entscheidung folgt der separat eingefrorenen P99-Schwelle und nicht unmittelbar der eingebauten `predict`-Grenze von `scikit-learn`.

Der Autoencoder rekonstruiert das standardisierte 128×3 -Fenster. Sein Score ist der mittlere quadratische Rekonstruktionsfehler über alle Zeitpunkte und Achsen. Die Architektur besitzt 507 trainierbare Parameter. Alle verdeckten Faltungsschichten verwenden ReLU, die Ausgabeschicht eine lineare Aktivierung. Faltungen und MaxPooling verwenden `padding = same`; die beiden UpSampling-Stufen verdoppeln jeweils die zeitliche Länge. Der TFLite-Interpreter wird mit einem Thread ausgeführt. Die Topologie ist in Tabelle 6-3 angegeben; Gewichte und Kalibrierung sind dem jeweiligen Profil zugeordnet.

Tabelle 6-3: Architektur des verwendeten Autoencoders

Stufe	Operation	Ausgabeform ohne Batchdimension
Eingabe	Standardisierte XYZ-Werte	128×3
Encoder 1	Conv1D, 8 Filter, Kernel 5; MaxPooling 2	64×8
Encoder 2	Conv1D, 4 Filter, Kernel 3; MaxPooling 2	32×4
Decoder 1	Conv1D, 4 Filter, Kernel 3; Upsampling 2	64×4
Decoder 2	Conv1D, 8 Filter, Kernel 3; Upsampling 2	128×8
Rekonstruktion	Conv1D, 3 Filter, Kernel 5; linear	128×3

6.5 Normaltraining, Schwellen und Exportprüfung

Das Profil wurde vor den Smartphoneversuchen aus vier normalen Aufnahmen von jeweils nominell 60 s bei 75 % PWM erstellt. Die ersten drei vollständigen Dateien dienen dem Training, die vierte der Validierung. Es entstehen 291 Trainings- und 97 Validierungsfenster. Der Scaler wird ausschließlich anhand der 37.248 XYZ-Punkte der Trainingsfenster angepasst. Die Aufteilung erfolgt nach vollständigen Aufnahmen; benachbarte Fenster derselben Datei werden nicht zufällig auf Training und Validierung verteilt.

Der Autoencoder wird mit Adam, Lernrate 0,001 und mittlerem quadratischem Fehler als Verlustfunktion trainiert. Batchgröße 32, maximal 100 Epochen und `shuffle = True` sind festgelegt. Python, NumPy und TensorFlow erhalten den Seed 42; deterministische TensorFlow-Operationen werden im Code angefordert. Das ersetzt keinen Nachweis identischer Trainingsläufe über beliebige Software- und Hardwarestände. Der Checkpoint speichert den kleinsten Validierungsverlust. EarlyStopping überwacht `val_loss` mit `patience =`

10, $\text{min_delta} = 10^{-6}$ und $\text{restore_best_weights} = \text{True}$. Die beste dokumentierte Epoche ist 100, ein vorzeitiger Abbruch erfolgt nicht. Damit ist keine vollständige Konvergenz nachgewiesen. Die abschließenden Verluste betragen etwa 0,25145 im Training und 0,27893 in der Validierung (B2; Trainingsprotokoll des Normalprofils). Abbildung 6-2 zeigt den gespeicherten Lernverlauf.

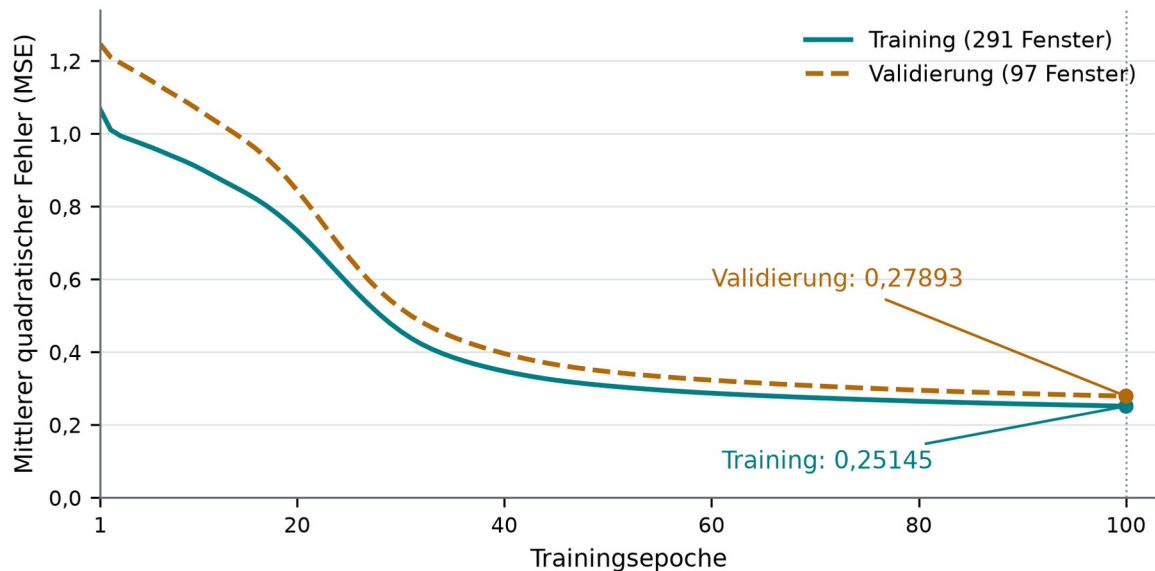


Abbildung 6-2: Trainings- und Validierungsverlust des Autoencoders. Gezeigt werden die protokollierten MSE-Werte der 100 Epochen, keine Erkennungsquoten. Der niedrigste Validierungsverlust liegt am Epochenlimit; vollständige Konvergenz ist damit nicht belegt. Quelle: B2, results/training_history.csv.

Die Trainings- und Validierungsverluste nehmen im dokumentierten Verlauf ab. Dies zeigt die Anpassung an die normalen Aufnahmen, belegt aber weder eine allgemeine Erkennungsqualität noch eine optimale Trainingsdauer. Insbesondere ersetzt die Validierungskurve keinen unabhängigen Test mit Ereignisreferenzen.

Für jede Methode wird der Schwellenwert als 99. Perzentil der Scores der separaten normalen Validierungsaufnahme bestimmt. Verwendet wird lineare Quantilinterpolation. Bei 97 sortierten Scores entspricht die Nullbasisposition $(97 - 1) \cdot 0,99 = 95,04$ einer Interpolation zwischen dem 96. und 97. geordneten Wert. Ein Alarm gilt bei $\text{Score} > \text{Schwelle}$. Die gleiche Perzentilregel bedeutet weder gleiche numerische Scorebereiche noch garantiert gleiche Fehlalarmraten auf neuen Daten.

Die vierte Normalaufnahme übernimmt zwei Rollen: Sie überwacht die Modellanpassung und liefert anschließend die Kalibrierungsscores. Modellwahl und Schwellenkalibrierung sind dadurch nicht vollständig unabhängig; eine Nutzung der späteren Smartphone-Testaufnahmen für diese Schritte ist nicht dokumentiert. Das P99 hängt bei 97 Werten von den beiden größten Validierungsscores ab. Die Schwelle ist daher eine nachvollziehbare Festlegung dieses Pilotprofils, aber kein statistisch abgesichertes 1%-Fehlalarmniveau.

Tabelle 6-4: Eingefrorene Schwellen der Smartphoneuntersuchung

Methode	Score	Schwelle
Autoencoder	MSE der standardisierten Rekonstruktion	0,477202599144
Isolation Forest	–score_samples	0,483354740160
RMS	RMS der standardisierten Werte	1,296361072562

Abbildung 6-3 zeigt die Verteilungen der 97 Kalibrierungsscores je Methode auf getrennten Scoreachsen. Die Autoencoder-Darstellung verwendet die ursprünglichen Keras-Rekonstruktionsfehler, aus denen die eingefrorene Schwelle stammt. Bei jeder Methode liegt eines dieser 97 Kalibrierungsfenster oberhalb der jeweiligen Schwelle. Das ist ein Befund auf den zur Grenzfestlegung verwendeten Daten, keine unabhängige Schätzung der Fehlalarmrate.

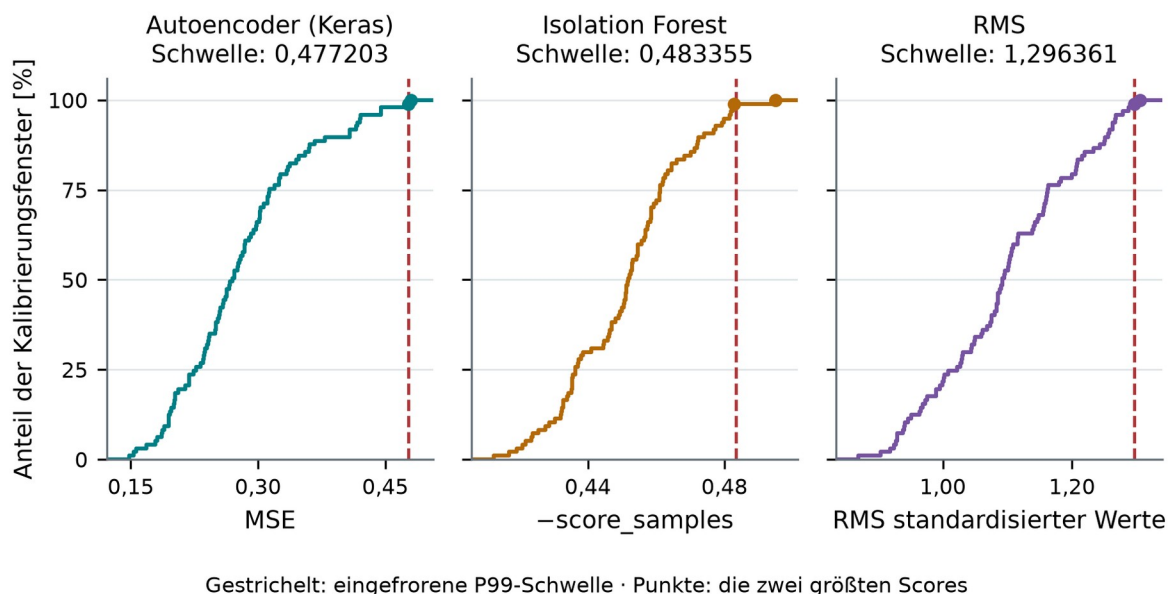


Abbildung 6-3: Verteilung der Kalibrierungsscores und eingefrorene P99-Schwellen. Treppendiagramme zeigen die empirischen Verteilungsfunktionen; senkrechte Linien kennzeichnen die separat linear interpolierten 99. Perzentile. Markiert sind jeweils die zwei größten Scores. Die Scoreachsen sind nicht untereinander kalibriert. Quelle: B2, results/validation_reconstruction_errors.csv (Keras); B1, bundle/validation_predictions.csv (Isolation Forest und RMS).

Die Autoencoder-Schwelle stammt aus der eingefrorenen Keras-Validierung. Der Float32-TFLite-Export wird auf denselben 97 Validierungsfenstern geprüft: Die größte absolute Rekonstruktionsabweichung beträgt $2,623 \cdot 10^{-6}$, die größte MSE-Abweichung $7,192 \cdot 10^{-8}$; keine Entscheidung ändert sich. Ein Validierungsfenster liegt oberhalb der P99-Grenze. Dieser Kalibrierungsbefund ist keine unabhängig gemessene Fehlalarmrate im späteren Betrieb. Modelle und Schwellen werden nach Sichtung der Smartphoneversuche nicht optimiert.

Die zugeordneten Artefakte umfassen 55.472 Byte für das Keras-Modell, 14.084 Byte für das Float32-TFLite-Modell, 2.955.725 Byte für Isolation Forest und 639 Byte für den Scaler. Die

TFLite-Datei ist damit rund 74,6 % kleiner als die Keras-Datei. Diese Dateigrößen beziehen sich auf unterschiedliche Serialisierungsformate und beschreiben weder den Prozessspeicher noch den elektrischen Energiebedarf. Der Trainingspfad ist in `src/calibrate_and_train.py`, `src/train_clean_autoencoder.py` und `src/train_autoencoder.py` dokumentiert; die Ausführung und Scorerichtung sind in `src/common_comparison.py` sowie den beiden Smartphone-Recordern festgelegt (Repositorystand im Anhang).

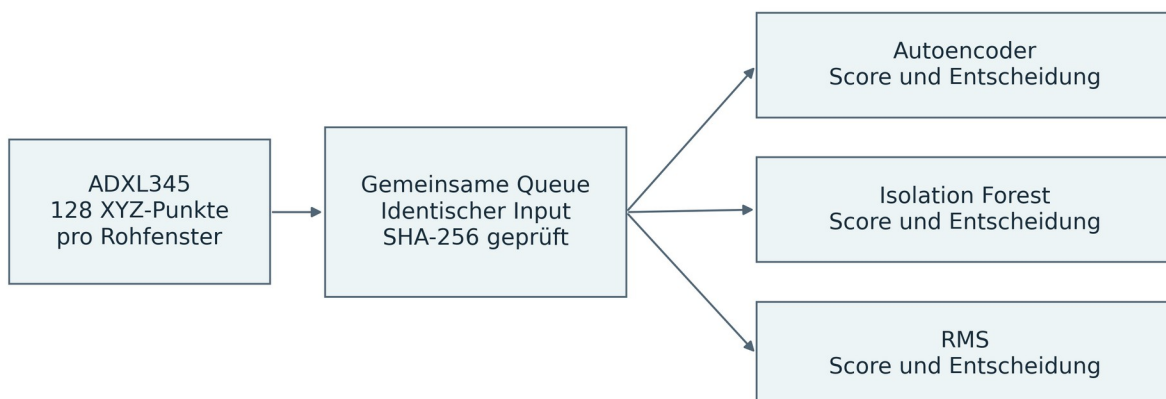
6.6 Einzelbetrieb, Replay und gemeinsame Live-Auswertung

Im Einzelbetrieb läuft jeweils nur eine Methode im Aufnahmeprozess. Erfassung, Qualitätsprüfung, Vorverarbeitung, Entscheidung und Protokollierung gehören dennoch zum Prozess. Vor Beginn werden 20 Aufwärmfenster verarbeitet. Es läuft keine Live-GUI. Die drei ausgewählten Einzelaufnahmen enthalten unterschiedliche physikalische Anregungen; ein identisches Smartphone-Modell allein macht verschiedene Anrufe nicht zu identischen Sensoreingaben.

Beim Offline-Replay werden alle drei Methoden auf die gleichen gespeicherten Rohfenster jeder Einzelaufnahme angewendet. Dadurch ist der Eingabevergleich exakt reproduzierbar. Die Reihenfolge der ersten Alarmfenster lässt sich vergleichen, die Replay-Laufzeit ist jedoch keine gemessene Live-Reaktion auf eine neue Smartphonevibration. Die erneute Auswertung erzeugt keine zusätzlichen unabhängigen physikalischen Ereignisse.

Im gemeinsamen Live-Lauf erfasst ein Sensorpfad die Rohdaten. Jedes vollständige Fenster wird an drei Worker-Threads im selben Prozess übergeben. Ein gemeinsamer Startmechanismus koordiniert die Auswertung; gleiche Startbedingungen garantieren keine identische CPU-Zuteilung. Für jedes Fenster werden Eingabehash, Dispatch-, Worker-Start- und Entscheidungszeit gespeichert. Die feste Reihenfolge der Zeilen im Journal ist nicht das Ergebnisranking. Dieses wird aus den gemessenen Entscheidungszeitpunkten berechnet.

Gemeinsame Live-Auswertung in einem Prozess



Einzelbetrieb und Replay werden als getrennte Vergleichsmodi ausgewertet.

Abbildung 6-4: Datenfluss des gemeinsamen Live-Vergleichs. Alle drei Methoden erhalten dasselbe Rohfenster; ihre Verarbeitung und Ausgabezeitpunkte bleiben getrennt protokolliert. Quelle: eigene Darstellung anhand der Implementierung und Journale in B1.

Tabelle 6-5: Zeitbegriffe und zulässige Interpretation

Größe	Bezugspunkte	Bedeutung
Verarbeitungszeit	Start bis Ende des jeweiligen Auswerteraufrufs	Vorverarbeitung, Modellbewertung und Score im instrumentierten Aufruf; kein einheitlicher reiner Modellkern
Fenster-bis-Entscheidung	Host-Abschluss des letzten Fensterpunkts bis Entscheidung	Einschließlich Übergabe und Scheduling; ohne vorherige Fensterfüllzeit
Erstes Alarmfenster	Erster Fensterindex mit Score > Schwelle	Vergleich der Eingabesensitivität bei identischen Fenstern
Relative Alarmausgabe	Erste Alarmausgabe einer Methode minus früheste Alarmausgabe	Beobachteter Ausgabevorsprung im selben Live-Lauf
Physische Erkennungsverzögerung	Entscheidung minus unabhängig gemessener Vibrationsbeginn	Mit den vorliegenden Bedienmarkern nicht bestimmbar

6.7 Ablauf, Bedienmarker und Annahmen

Das Versuchsprotokoll sieht mindestens 30 s Ruhe vor der Anrufaufforderung und mindestens 15 s Nachlauf nach der gemeldeten Beendigung vor. Start, Protokollierung, Auswertung und Diagrammerstellung erfolgen skriptgestützt; die Person am Aufbau löst den Anruf manuell aus. Anläufe mit unklarer Abstimmung bleiben als solche erhalten und werden getrennt dokumentiert. Die KI-Unterstützung des Arbeitsablaufs ist im Abschnitt KI-Unterstützung und fachliche Verantwortung im Anhang offengelegt.

Der Marker `call_requested` bezeichnet die protokollierte Aufforderung, nicht den tatsächlichen Beginn der Smartphonevibration. Ebenso ist die Rückmeldung zum Ende keine automatische Messung des mechanischen Abklingens. Die geplanten ungefähr 5 s Vibration werden nicht als tatsächlich gemessene Dauer übernommen. Der Zeitraum zwischen den beiden Bedienmarkern umfasst auch Reaktions-, Verbindungs- und Rückmeldezeit. Die Abbildungen schattieren diesen Bedienzeitraum ausdrücklich, nicht ein verifiziertes Störungsintervall.

Fenster werden anhand ihrer Anfangs- und Endzeit in vor der Aufforderung, zwischen den Markern, nach der Rückmeldung und markerübergreifend eingeteilt. Markerübergreifende Fenster bleiben gesondert erhalten. Ohne unabhängige physische Referenz ist die Anzahl markierter Fenster eine deskriptive Alarmzahl. Elf Alarmfenster bedeuten weder elf unabhängige Störungen noch automatisch eine bessere Erkennungsquote als sechs Alarmfenster.

Tabelle 6-6: Annahmen und offene Nachweise

Annahme / Größe	Vorliegender Nachweis	Konsequenz
Gleiche Eingaben	Gemeinsamer Rohstrom und Hash je Fenster im Live-Vergleich; identische Rohdateien im Replay	Direkter Methodenvergleich innerhalb dieser Modi möglich
Gleiche Anrufe im Einzelbetrieb	Keine unabhängige Gleichheitsmessung	Keine direkte Rangfolge aus getrennten Startzeiten
Übertragbarkeit des Normalprofils	Vorheriges Normaltraining; endgültige Smartphonegeometrie nicht vollständig bestätigt	Ergebnisse bleiben auf den Pilotaufbau begrenzt
Physischer Beginn / Ende	Nur Bedienmarker und Nutzerrückmeldung	Keine absolute Fehler-Erkennungsverzögerung oder bestätigte F1-Auswertung
Elektrischer Verbrauch	Keine Spannungs-/Strommessung	CPU und RAM werden nicht in Watt oder Wh umgerechnet
Wiederholbarkeit	Ein ausgewählter Anruf je Einzelmethode, ein gemeinsamer Lauf	Keine Signifikanz- oder allgemeine Zuverlässigkeitsaussage

6.8 Dateien, Visualisierung und Nachvollziehbarkeit

Die Recorder speichern raw.csv, decisions.csv, run.json und cues.jsonl; der gemeinsame Lauf zusätzlich resources.csv. Laufdateien verknüpfen Messungen, Entscheidungen, Modellpaket sowie Sensor- und PWM-Rücklesungen. Die Auswertung nutzt abgeschlossene Journale. protocol.md dokumentiert dagegen die Versuchsplanung. Die Versuche liefen ohne Live-GUI, deren Zusatzlast nicht bestimmt wurde. Abbildungen wurden anschließend aus den gespeicherten Daten erzeugt.

7 Evaluation

7.1 Umfang und Auswahl der Aufnahmen

Die Smartphoneuntersuchung vom 19. September umfasst sechs Einzelanläufe und einen gemeinsamen Live-Lauf. Drei Einzelanläufe wurden nach protokollierten Abstimmungsproblemen durch spätere Aufnahmen ersetzt; alle bleiben im Archiv. AE03, IF02 und RMS01 bilden die ausgewählte deskriptive Auswertung. Ein formal vorab festgelegter, vollständig ergebnisunabhängiger Auswahlmechanismus ist damit nicht nachgewiesen. Die Auswahl und die geringe Ereigniszahl begrenzen die Übertragbarkeit. Auch die ausgewählten Läufe enthalten nur Bedienmarker und keine unabhängig verifizierte physikalische Ereigniszeit.

Tabelle 7-1: Erhaltene Anläufe und Auswertungsrolle

Kurzname	Vollständige Fenster	Alarmfenster	Rolle
AE01	170	30	Früher Anlauf; möglicher zu früher Anruf, wiederholt
AE02	158	12	Erneuter Abstimmungsversuch; ein ungültiges Fenster, wiederholt
AE03	184	11	Ausgewählter Autoencoder-Einzelprozess
IF01	154	7	Anrufkoordination unklar/verpasst, wiederholt
IF02	196	6	Ausgewählter Isolation-Forest-Einzelprozess
RMS01	234	6	Ausgewählter RMS-Einzelprozess
Gemeinsam01	197 je Methode	AE 11 / IF 7 / RMS 6	Gemeinsamer Live-Vergleich

Technische Funktionsprüfungen im Ordner `technical_checks` sind nicht Teil dieser Ergebnisstichprobe. AE03, IF02 und RMS01 entsprechen den vollständigen Ordernamen `autoencoder_awaiting_call_03`, `isolation_forest_awaiting_call_02` und `rms_awaiting_call_01`. Gemeinsam01 liegt unter `joint_trials/all_methods_01`. Diese Zuordnung verhindert die Verwechslung von Wiederholungsanläufen und ausgewerteten Aufnahmen.

7.2 Datenintegrität und Reproduzierbarkeit

Die drei ausgewählten Einzelaufnahmen enthalten zusammen 78.843 XYZ-Punkte, 614 vollständige Fenster und 251 gespeicherte Restpunkte. Der gemeinsame Lauf enthält 25.270 XYZ-Punkte, 197 vollständige Fenster und 54 Restpunkte. Damit liegen 811 verschiedene vollständige Rohfenster der vier ausgewählten Aufnahmen vor. Der gemeinsame Lauf erzeugt daraus 591 Entscheidungen; zusammen mit den 614 Einzelentscheidungen sind es

1.205 ursprüngliche Live-Entscheidungen. Diese Anzahl ist keine Zahl unabhängiger Versuche.

In diesen vier Aufnahmen wurden keine ungültigen Modellfenster, verworfenen Fenster, Gap- oder Overrun-Ereignisse und keine Sättigungsflags protokolliert. Im gemeinsamen Lauf verblieben nach Abschluss keine vollständigen unverarbeiteten Fenster. Solche Nullbefunde belegen die protokollierte Integrität, nicht die Abwesenheit aller denkbaren unbekannten Messfehler.

Alle 614 ursprünglichen Einzelentscheidungen lassen sich beim Replay exakt reproduzieren; die größte Scoreabweichung beträgt 0. Auch die 591 Entscheidungen des gemeinsamen Laufs wurden aus denselben Rohfenstern exakt reproduziert. Für alle 197 gemeinsamen Fenster stimmen die Eingabehashes der drei Methoden überein. Zusätzlich wurden vor dem gemeinsamen Lauf 1.842 Auswertungen der drei Methoden auf den 614 gespeicherten Einzelfenstern abgeglichen. Diese Softwareprüfungen bestätigen Konsistenz, nicht zusätzliche unabhängige Erkennungsereignisse.

Tabelle 7-2: Ausgewählte Aufnahmen und Verteilung der gültigen Fenster

Aufnahme	Rohpunkte / Rest	Vor Marker	Zwischen Markern	Nach Marker / übergreifend
AE03	23.617 / 65	83	52	47 / 2
IF02	25.170 / 82	76	59	59 / 2
RMS01	30.056 / 104	83	109	40 / 2
Gemeinsam01	25.270 / 54	77	79	39 / 2

7.3 Einzelbetrieb und Vergleich auf identischen Replay-Fenstern

Im ausgewählten Einzelbetrieb markiert der Autoencoder elf Fenster, Isolation Forest sechs und RMS sechs. Die Alarmer liegen jeweils vollständig zwischen den Bedienmarkern. Vor der Anrufaufforderung und nach der Enderückmeldung treten in diesen ausgewählten Läufen keine Alarmer auf. Weil Anrufe und Zeitverläufe voneinander abweichen, sind diese Zahlen allein weder ein Geschwindigkeitsranking noch eine faire Erkennungsquote.

Der Offline-Vergleich wertet deshalb jede dieser drei Rohaufnahmen mit allen drei eingefrorenen Methoden aus. Auf AE03 markieren alle Methoden erstmals Fenster 97. Auf IF02 markiert der Autoencoder Fenster 93, die beiden Baselines erst Fenster 94. Auf RMS01 beginnen alle drei bei Fenster 155. Der Autoencoder ist somit in diesen drei Aufnahmen nie später im ersten Alarmfenster, einmal früher und zweimal gleichauf. Die drei Befunde stammen von drei verschiedenen Anrufen; zusätzliche Verallgemeinerung erfordert mehr unabhängige Wiederholungen.

Tabelle 7-3: Replay derselben Rohfenster; erstes Alarmfenster und gesamte Alarmfenster

Aufnahme	Autoencoder	Isolation Forest	RMS
AE03	erstes 97; 11 Alarme	erstes 97; 6 Alarme	erstes 97; 8 Alarme
IF02	erstes 93; 9 Alarme	erstes 94; 6 Alarme	erstes 94; 6 Alarme
RMS01	erstes 155; 9 Alarme	erstes 155; 6 Alarme	erstes 155; 6 Alarme

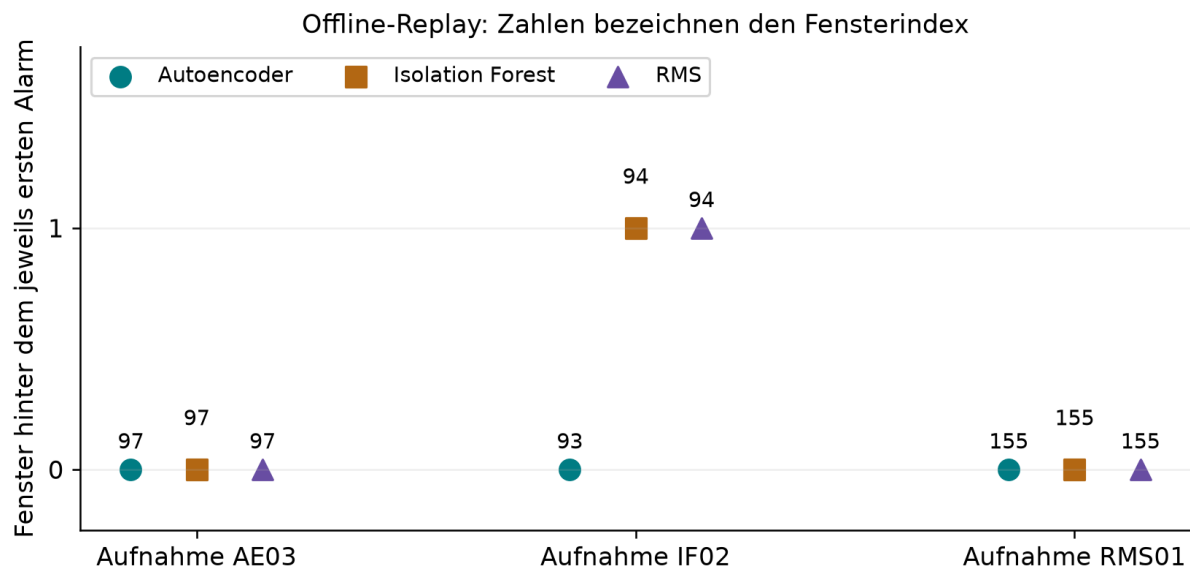


Abbildung 7-1: Erstes Alarmfenster im Offline-Replay. Die Zahlen an den Punkten sind Fensterindizes; die vertikale Achse zeigt den Abstand zum frühesten Alarmfenster derselben Aufnahme. Eine Null bedeutet Gleichstand auf Fensterebene, keine gleichzeitig gemessene Live-Ausgabe. Quelle: B1, replay_v1/summary.json.

7.4 Gemeinsamer Live-Lauf: Signal und Alarmreihenfolge

Im gemeinsamen Lauf erhalten alle Methoden dieselben 197 Fenster. Die Anruaufforderung liegt 48,019 s, die protokollierte Enderückmeldung 97,322 s nach dem Aufnahmestart. Dazwischen befinden sich 79 vollständige Fenster. Der Autoencoder markiert elf, Isolation Forest sieben und RMS sechs davon. In den 77 Fenstern vor der Aufforderung, den 39 Fenstern nach der Rückmeldung und den zwei markerübergreifenden Fenstern gibt es bei keiner Methode einen Alarm. Diese Nullwerte beschreiben die gespeicherte Folge, nicht eine nachgewiesene allgemeine Fehlalarmrate von 0 %.

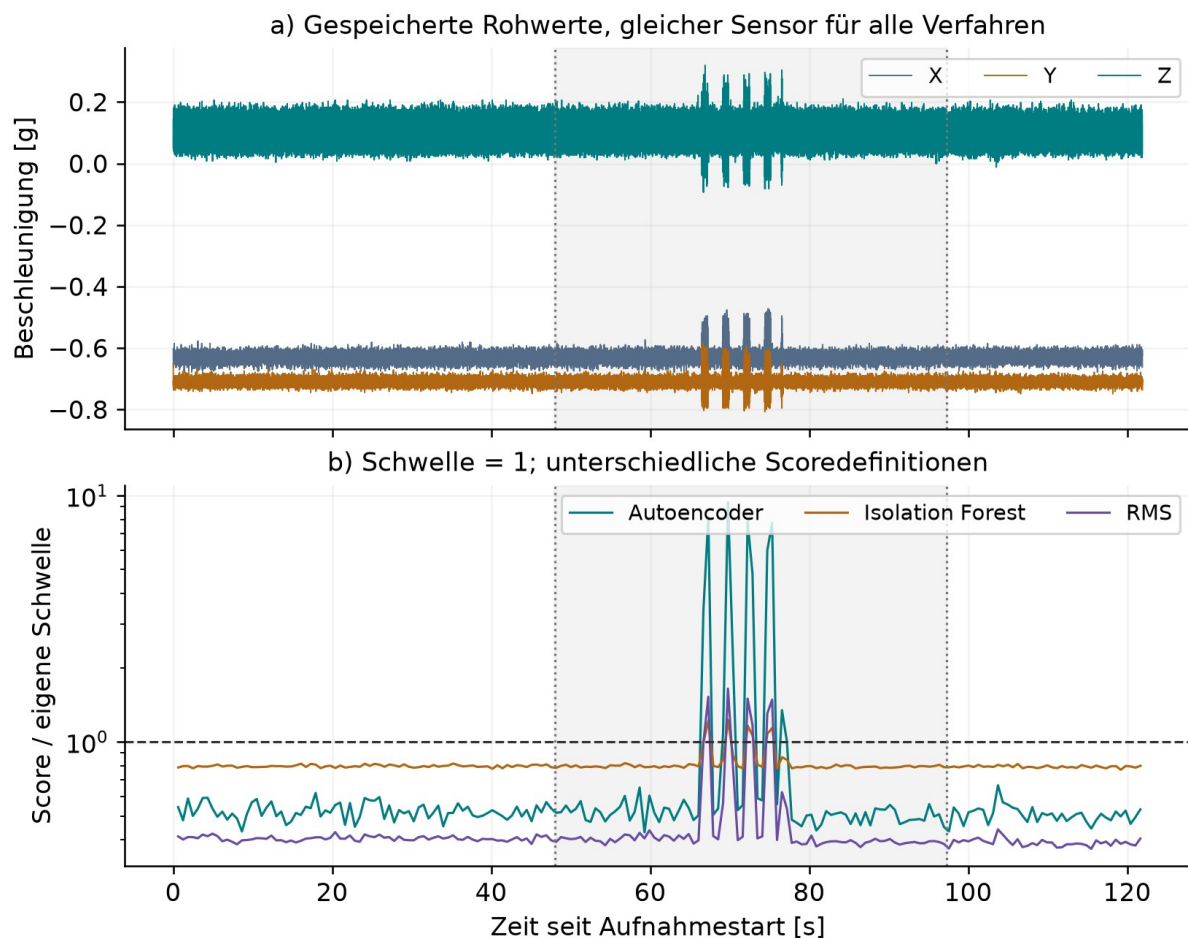


Abbildung 7-2: Gemeinsamer Rohdaten- und Scoreverlauf. Graue Bereiche liegen zwischen den Bedienmarkern, nicht zwischen unabhängig gemessenem Vibrationsbeginn und -ende. Die Scoreachse ist logarithmisch und auf die jeweilige Schwelle normiert; der Wert 1 kennzeichnet die Alarmgrenze. Quelle: B1, Gemeinsam01.

Tabelle 7-4: Erste Alarime im gemeinsamen Live-Lauf

Methode	Erstes Alarmfenster	Rang des Fensters	Rang der Ausgabe	Abstand zur ersten Ausgabe [ms]
Autoencoder	108	1, gleichauf mit IF	1	0,000
Isolation Forest	108	1, gleichauf mit AE	2	19,061
RMS	109	2	3	615,659

Der Autoencoder und Isolation Forest überschreiten ihre Schwelle erstmals auf demselben Fenster 108. Die Autoencoder-Entscheidung liegt um 19,061 ms früher vor. RMS benötigt wesentlich weniger Rechenzeit, markiert dieses Fenster aber noch nicht als auffällig. Seine erste Alarmausgabe erfolgt für Fenster 109 und damit 615,659 ms nach derjenigen des Autoencoders. Die Trennung zwischen erstem Alarmfenster und Ausgabezeitpunkt ist entscheidend: Das spätere RMS-Ergebnis entsteht hier nicht durch langsame RMS-Berechnung, sondern überwiegend durch die spätere Schwellenüberschreitung.

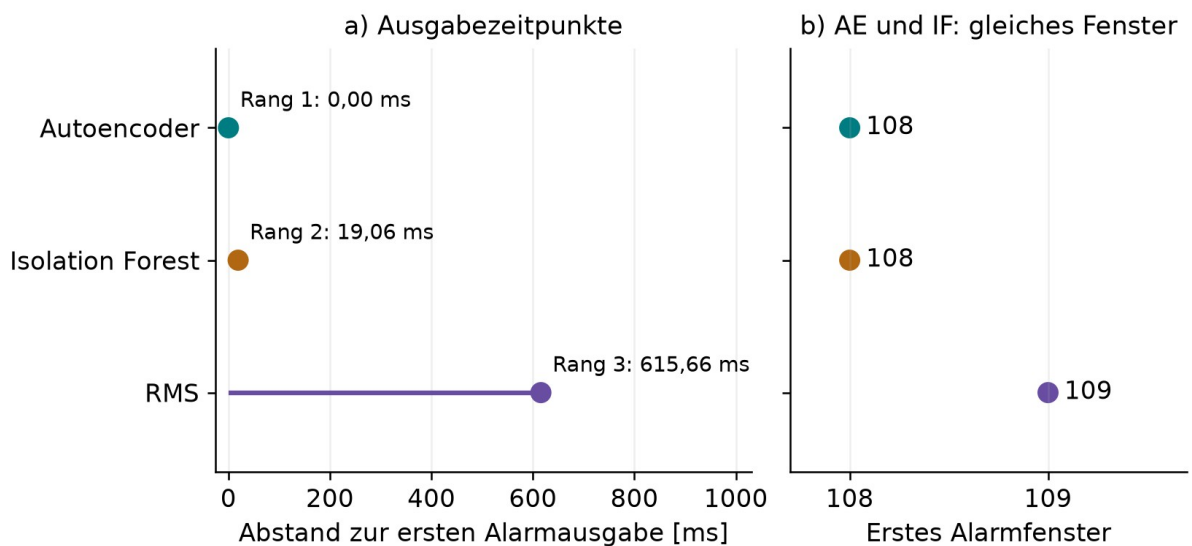


Abbildung 7-3: Nummerierte Reihenfolge der ersten Alarmausgaben im gemeinsamen Lauf und zugehörige Fensterindizes. Null ist der Zeitpunkt des ersten Autoencoder-Alarms, nicht der physische Störungsbeginn. AE und IF sind auf Fensterebene gleichauf; die Ausgabezeitpunkte unterscheiden sich. Quelle: B1, Entscheidungen von Gemeinsam01.

Die relative Zeitdifferenz wird aus der monotonen Uhr berechnet: $\Delta t_m = (t_{m,erste\ Alarmausgabe} - \min_k t_{k,erste\ Alarmausgabe}) / 10^6$ in Millisekunden. Dabei werden die tatsächlichen Entscheidungszeitpunkte verwendet, nicht die Reihenfolge der Methodenliste oder der CSV-Zeilen. Die Daten belegen den Ausgabevorsprung des Autoencoders für diesen Lauf. Eine absolute Reaktionszeit ab Beginn der Smartphonevibration ist mangels unabhängiger Referenz nicht bestimmbar.

7.5 Warum unterscheiden sich die ersten Alarme?

Tabelle 7-5: Unmittelbarer Schwellenvergleich auf dem gemeinsamen Fenster 108

Methode	Score	Eingefrorene Schwelle	Entscheidung
Autoencoder	1,631938754	0,477202599	Alarm
Isolation Forest	0,484321113	0,483354740	Alarm
RMS	1,282930084	1,296361073	Normal

Auf Fenster 108 liegt der Autoencoder-Fehler deutlich oberhalb seiner Normalgrenze. Isolation Forest liegt nur knapp darüber. RMS liegt knapp darunter und überschreitet seine Schwelle erst in Fenster 109 mit einem Score von etwa 1,97525. Fenster 107 ist noch bei allen drei Methoden unauffällig. Die Werte erklären die protokollierte Rangfolge direkt und ohne eine nachträgliche Anpassung der Schwellen.

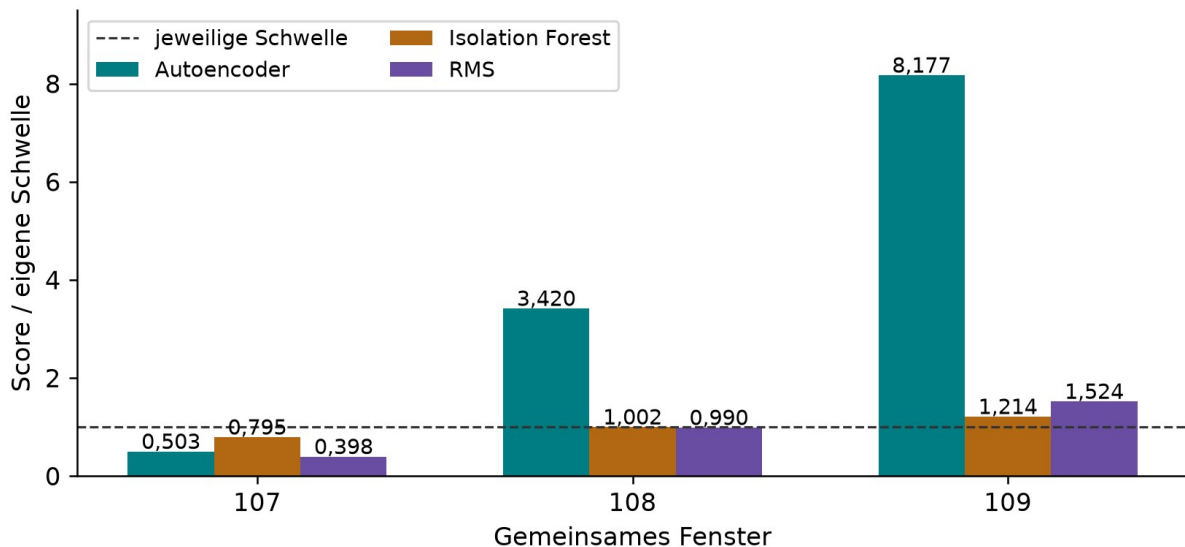


Abbildung 7-4: Schwellenübergang der Fenster 107 bis 109. Die Normierung erleichtert das Ablesen der jeweiligen Grenzüberschreitung; die Quotienten sind weder Wahrscheinlichkeiten noch untereinander kalibrierte Störungsschweregrade. Quelle: B1, Gemeinsam01.

Als mögliche Erklärung nutzt der Autoencoder Unterschiede im standardisierten Fenster, die ein einzelner aggregierter quadratischer Abweichungskennwert nicht getrennt darstellt. Der vorliegende Vergleich isoliert jedoch weder die zeitliche Repräsentation noch einen einzelnen Einflussfaktor. Unterschiede der Scorefunktion, der trainierten Modelle und der kleinen Kalibrierungstichprobe können gemeinsam zur beobachteten Schwellenüberschreitung beitragen. Eine kausale Erklärung benötigt gesonderte kontrollierte Vergleiche.

Die Verarbeitung des ersten Alarmfensters dauert beim Autoencoder etwa 2,789 ms und beim Isolation Forest etwa 21,733 ms. Der Ausgabeabstand von 19,061 ms entsteht im selben gemeinsam gestarteten Fenster und ist mit diesem Aufwandsunterschied vereinbar; Scheduling und weitere Übergaben sind ebenfalls enthalten. RMS benötigt im ersten eigenen Alarmfenster nur etwa 0,183 ms. Rechenzeit und frühzeitige Anomalieausgabe sind somit unterschiedliche Bewertungskriterien.

7.6 Rechenaufwand und Verhalten des Raspberry Pi

Die Ressourcenmessung bezieht sich auf den jeweils ausgeführten Smartphone-Recorder. Einzelprozesse und gemeinsamer Prozess unterscheiden sich in aktiven Methoden, geladenen Bibliotheken und Konkurrenz um Rechenzeit. Identische Hardware bedeutet daher keine identischen CPU-, RAM- oder Latenzwerte. Die folgenden Ergebnisse bleiben den jeweiligen Prozessgrenzen und Messdefinitionen zugeordnet.

Tabelle 7-6: Verarbeitungszeit im jeweiligen Auswerterauf; ms

Methode	Einzelbetrieb Median	Einzelbetrieb P95	Gemeinsam Median	Gemeinsam P95
Autoencoder	0,459	0,585	2,040	5,571
Isolation Forest	23,429	32,061	23,237	31,527
RMS	0,285	0,352	0,264	0,317

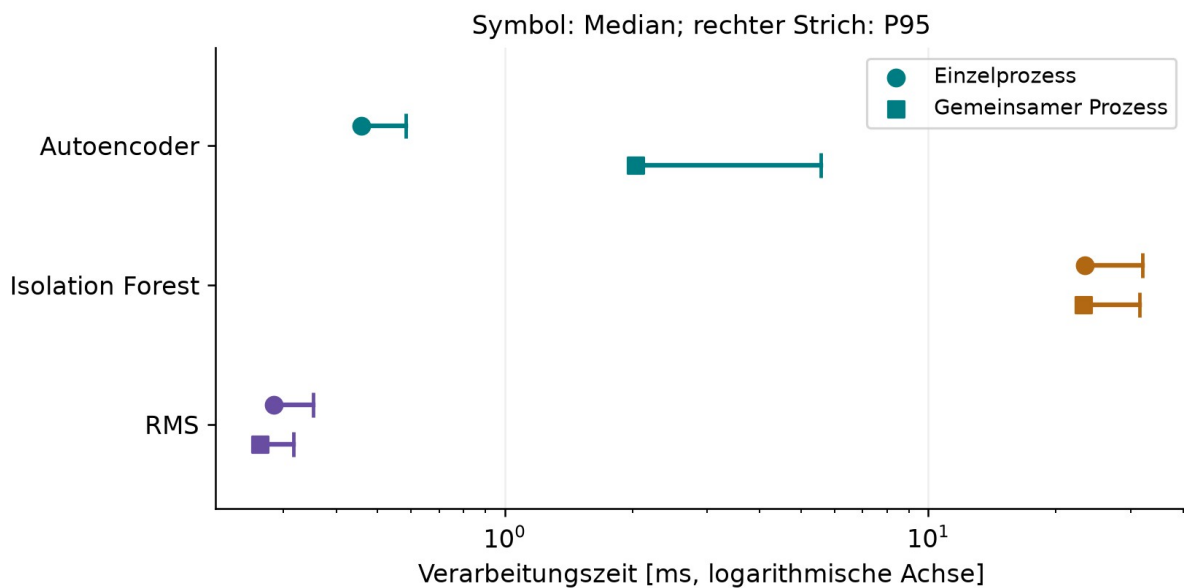


Abbildung 7-5: Verarbeitungszeiten im Einzelbetrieb und im gemeinsamen Prozess. Symbole zeigen Median, rechte Striche P95; die Zeitachse ist logarithmisch. Erfasst wird der instrumentierte Auswerterauf; einschließlich seiner Vorverarbeitung, nicht eine einheitlich isolierte Modellkernzeit. Quelle: B1, ausgewählte decisions.csv.

Der Isolation Forest benötigt in beiden Modi die meiste Verarbeitungszeit. RMS ist am schnellsten berechnet. Der Autoencoder liegt dazwischen; seine gemessene Zeit steigt im gemeinsamen Prozess. Diese Beobachtung ist mit zusätzlicher Konkurrenz und anderem Scheduling vereinbar, ist aber ohne kontrollierte Wiederholungsmatrix keine isolierte Ursache-Wirkungs-Messung. Die Einzelaufnahmen unterscheiden sich außerdem in Dauer und Ereignisverlauf.

Die CPU-Kennzahl ist der Median der im jeweiligen Lauf protokollierten Prozessmessungen; 100 % entsprechen einem logischen Kern. Sie umfasst den gesamten Aufnahmeprozess einschließlich Erfassung und Protokollierung. Der RSS-Wert ist der größte während der protokollierten Fenster abgetastete residente Prozessspeicher. Er ist weder der gesamte RAM des Raspberry Pi noch zwingend der höchste Wert während Import, Modellstart oder Prozesslebensdauer. Ein Punkt zwischen zwei Messproben kann unbemerkt bleiben.

Tabelle 7-7: Prozessressourcen der ausgewählten Smartphoneaufnahmen

Betriebsart	CPU-Median [% eines Kerns]	Größter abgetasteter RSS [MiB]	Umfang
Autoencoder allein	9,72	48,80	184 Fenster
Isolation Forest allein	12,98	167,11	196 Fenster
RMS allein	9,69	39,31	234 Fenster
Alle drei gemeinsam	13,04	174,42	197 gemeinsame Fenster

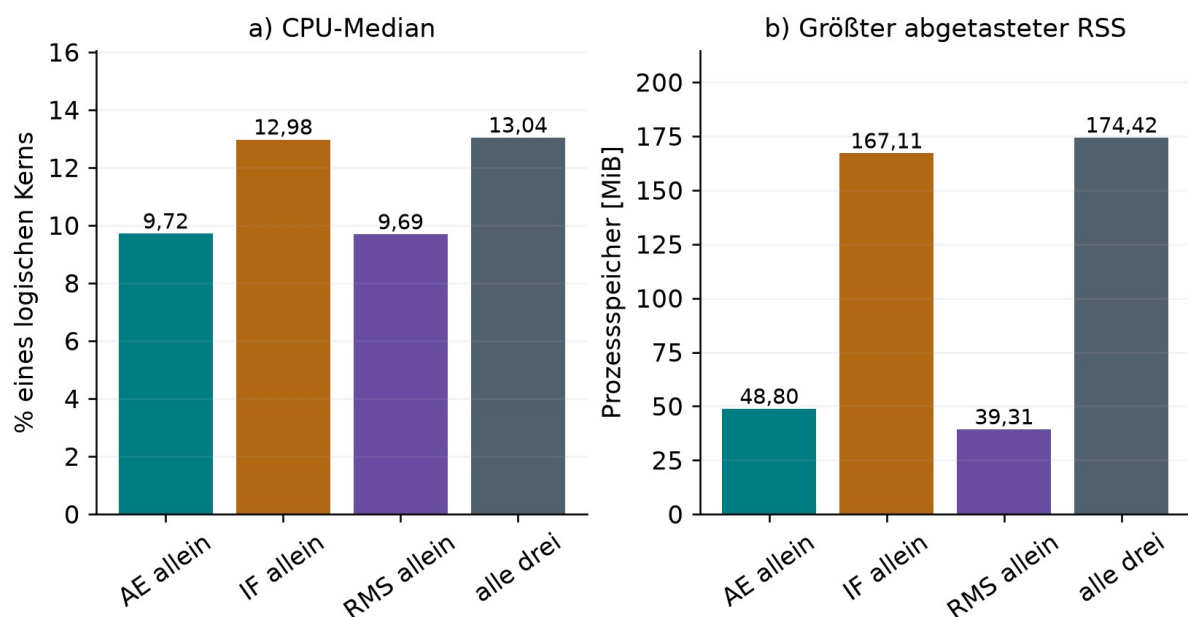


Abbildung 7-6: CPU und RAM im Einzelbetrieb und gemeinsamen Live-Prozess. Es handelt sich um Prozessmessungen mit unterschiedlichen Laufdauern; die gemeinsame Auslastung darf nicht pro Methode aufgeteilt oder aus den Einzelwerten addiert werden. Quelle: B1, decisions.csv und resources.csv.

Die gemeinsame Speichernutzung liegt in diesem Lauf etwas oberhalb des Isolation-Forest-Einzelprozesses, nicht bei der Summe aller Einzelprozesse. Im gemeinsamen Prozess werden Teile der Erfassung, Laufzeitumgebung und geladenen Bibliotheken geteilt. Eine genaue Zuordnung einzelner Speicheranteile wurde nicht instrumentiert. Die Ergebnisse zeigen im untersuchten Umfang eine Ausführung ohne protokollierten Fensterrückstand oder Datenverlust; sie sind kein unbegrenzter Dauerbetriebsnachweis.

Im gemeinsamen Lauf beträgt das empirische P99 der Zeit vom Host-Abschluss des letzten Fensterpunkts bis zur Entscheidung 9,156 ms für AE, 45,008 ms für IF und 8,793 ms für RMS. Die Maxima sind 12,205 ms, 83,893 ms und 11,455 ms. Die protokollierte Erfassungsrate von 207,298 XYZ-Punkten/s ergibt für das Kriterium aus Abschnitt 1.4 ein Fensterbudget von 617,468 ms; der nominelle Wert 640 ms ist eine Konfigurationsangabe. Zusätzlich lassen sich 196 tatsächliche Folgefensterankünfte je Methode prüfen. Ihre Intervalle liegen zwischen 613,663 und 621,993 ms. Keine der jeweils 196 Entscheidungen erfolgt nach der nächsten

Fensterankunft. Das letzte Fenster besitzt kein Folgefenster und bleibt aus diesem Nenner ausgeschlossen. Damit ist das technische Latenzziel im beobachteten Lauf erfüllt; eine harte Echtzeitgarantie folgt daraus nicht. Fensterfüllzeit und physische Erkennungsverzögerung sind davon getrennt (B1, run.json und decisions.csv).

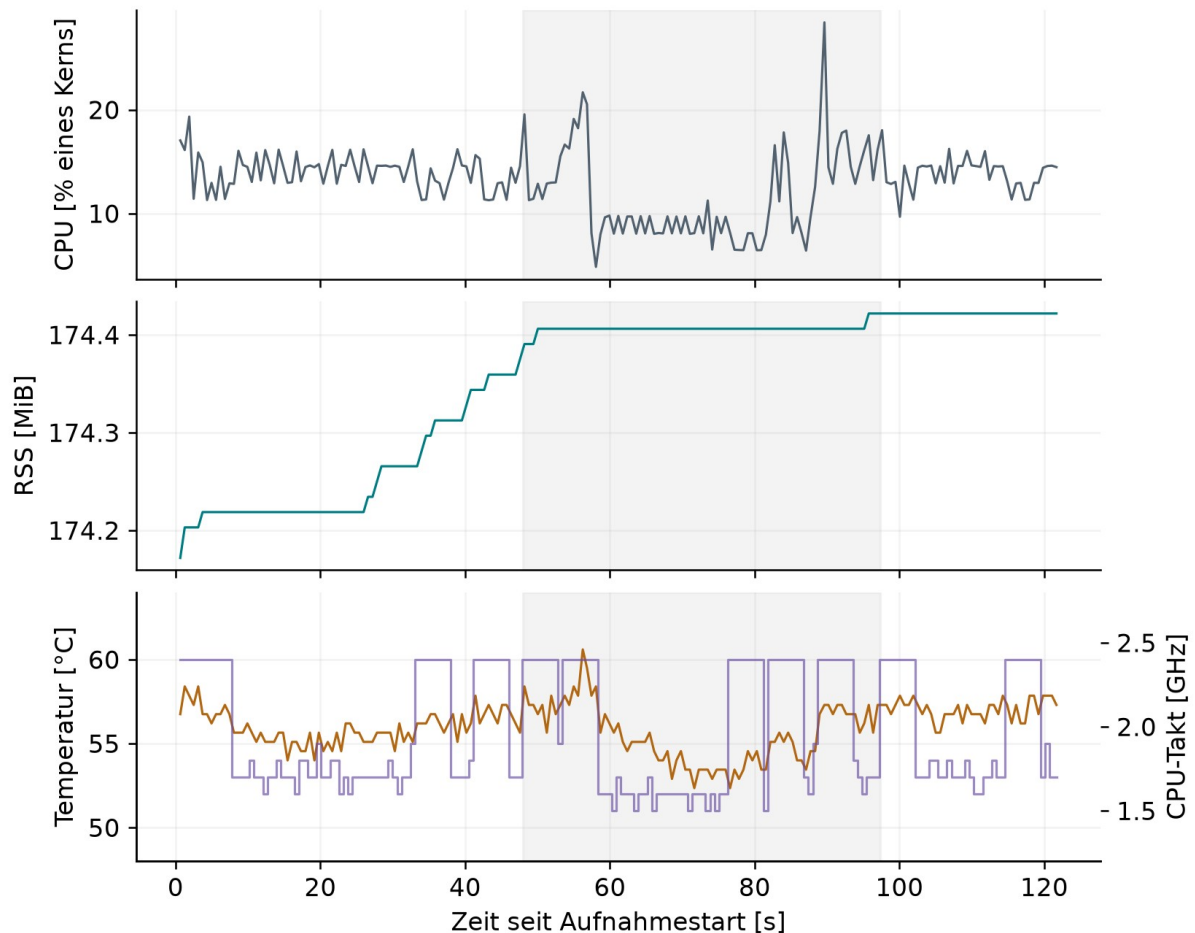


Abbildung 7-7: Zeitlicher Ressourcenverlauf des gemeinsamen Prozesses. Grau: Zeitraum zwischen Bedienmarkern. Temperatur und CPU-Takt sind zusätzliche Betriebsdiagnosen, keine Leistungs- oder Energiemessung. Quelle: B1, Gemeinsam01/resources.csv.

Im gemeinsamen Lauf wurden Temperaturen von 52,35 bis 60,60 °C und CPU-Frequenzen von 1,5 bis 2,4 GHz protokolliert. Über die ausgewählten Einzelprozesse reichen die Temperaturproben von 51,80 bis 61,15 °C. Aus wechselnder CPU-Frequenz allein folgt kein Nachweis thermischer Drosselung. Für diese Reihe liegt keine vollständige unabhängige Drosselungsanalyse vor. Ein unauffälliger kurzer Versuch schließt spätere Temperatur- oder Speicherprobleme nicht aus.

Unter Verbrauch wird in dieser Auswertung ausschließlich der beobachtete Rechen- und Speicherbedarf verstanden. Elektrische Leistung in Watt, Stromaufnahme und Energie in Wh wurden nicht gemessen. CPU-Auslastung ist kein hinreichender Ersatz dafür. Auch die externe 12-V-Lüfterversorgung gehört nicht automatisch zum erfassten Raspberry-Pi-Prozessbedarf. Eine Aussage wie „der Autoencoder verbraucht weniger elektrische Energie“ wird deshalb nicht aus diesen Daten abgeleitet.

7.7 Forschungsfragen und Nachweisstand

Tabelle 7-8: Beantwortung der Forschungsfragen und Nachweisgrenzen

Gegenstand	Beobachtung	Bewertung
FF1: technische Auswahl	Raspberry Pi, ADXL345 und drei lokale Verfahren umgesetzt	Technische Eignung im Pilotbetrieb gezeigt; vollständige Messqualität bleibt offen
FF2: Alarmverhalten	Gemeinsam: AE/IF erstmals Fenster 108, RMS 109; AE-Ausgabe 19,1 ms vor IF und 615,7 ms vor RMS	Deskriptiver Ausgabevorsprung in einem gemeinsamen Ereignis; kein allgemeiner Qualitätsnachweis
FF3: Zeitbudget und Ressourcen	P99 unter 617,468 ms; je Methode 0/196 Überschreitungen der Folgefensterankunft; CPU 13,04 %, RSS 174,42 MiB gemeinsam	Technisches Latenzziel im beobachteten Lauf erfüllt; keine harte Echtzeit- oder Dauerbetriebsgarantie
FF4: Aussagekraft	Gleiche Fensterhashes und reproduzierte Entscheidungen; physikalische Ereignisreferenz und Wiederholungen begrenzt	Technische Vergleichbarkeit belegt; F1, bestätigte Fehlalarmrate und absolute Erkennungsverzögerung nicht bestimmbar

Nachvollziehbarkeit, gleiche Eingaben, Datentrennung und Ressourcenmessung werden durch Dateien, Hashes und getrennte Vergleichsmodi gestützt. Vollständig verifizierte Messqualität, Drehzahl, physikalische Referenz und reproduzierbare Anregung bleiben teilweise offen. Die Bewertung bezieht sich auf den in Kapitel 4 offengelegten Rahmen der explorativen Studie. Sie ist kein nachträglicher Nachweis einer vorab registrierten bestätigenden Untersuchung.

Tabelle 7-9: Rückbindung an die Anforderungen aus Kapitel 4

Anforderung	Stand der Smartphoneuntersuchung	Abgrenzung / Nachweis
A1 Messqualität	Teilnachweis	Integritätsflags geprüft; Bandbreite, Signal-Rausch-Abstand und Sensortakt nicht vollständig unabhängig abgesichert
A2 Datentrennung	Im dokumentierten Umfang erfüllt	Drei normale Trainingsdateien, eine getrennte normale Validierungsdatei, spätere Smartphoneaufnahmen
A3 Einheitlicher Vergleich	Im direkten Eingabevergleich erfüllt	Identische Rohfenster im Replay und gemeinsamen Live-Lauf; gleiche P99-Regel und Markerzuordnung
A4 Lokale Auswertung	Im dokumentierten Umfang erfüllt	Entscheidungen lokal auf dem Pi; kein Cloud-Dienst für die Inferenz
A5 Nachvollziehbarkeit	Im dokumentierten Umfang erfüllt	Laufkennungen, Rohdaten, Bündel-/Dateihashes und Softwareprovenienz vorhanden
A6 Modellkompatibilität	Im dokumentierten Umfang erfüllt	Keras-/TFLite-Prüfung und exakte Reproduktion der Live-Entscheidungen
A7 Latenzziel	Im beobachteten Lauf erfüllt	P99 unter dem laufbezogenen Fensterbudget; je Methode 0/196 Überschreitungen gegenüber tatsächlichen Folgefenstern
A8 Speicher und Rückstand	Teilnachweis	Kurze Läufe ohne protokollierten Abbruch/Rückstand; kein Dauerbetriebs- oder vollständig kontrollierter Lastnachweis
A9 Alarmverhalten	Deskriptiv ausgewertet	Alarmzahlen, erste Fenster und relative Ausgabezeitpunkte berichtet; keine Konfusionsmatrix ohne unabhängige Referenz
A10 Anregung und Referenz	Teilnachweis	Bedienmarker vorhanden; physikalischer Beginn und Ende sowie finale Smartphonegeometrie nicht unabhängig bestätigt
A11 Reproduzierbarkeit	Teilnachweis	Protokoll und Daten erhalten; Smartphonegeometrie und Wiederholungsgleichheit nicht vollständig bestätigt
A12 Live-GUI	Nicht Gegenstand dieser Messung	GUI vorhanden, Aufnahme ohne GUI; Zusatzaufwand nicht bestimmt
A13 Geordnetes Starten/Stoppen	Im dokumentierten Umfang erfüllt	Abgeschlossene Journale, gespeicherte Reste und keine verbliebenen vollständigen Fenster

8 Diskussion

8.1 Alarmverhalten und Ausgabezeitpunkte

Der Smartphoneversuch zeigt eine Reaktion aller drei Verfahren auf den aufgezeichneten Verlauf: In den ausgewählten Läufen treten Alarme zwischen den Bedienmarkern auf. Die gemeinsame Aufnahme erlaubt dabei einen Vergleich auf exakt denselben Rohfenstern. Ohne unabhängige physikalische Zeitreferenz lässt sich jedoch nicht jedes markierte Fenster eindeutig der Vibration, ihrem Nachschwingen oder einer sonstigen Abweichung zuordnen. Die Alarmzahlen sind deshalb kein Ersatz für Precision, Recall oder F1.

Im gemeinsamen Ereignis markieren AE und IF erstmals dasselbe Fenster, die AE-Ausgabe liegt jedoch rund 19,1 ms früher vor. RMS berechnet seinen Score schneller, überschreitet seine Schwelle aber erst im Folgefenster. Im Replay der drei weiteren ausgewählten Ereignisse beginnt der AE einmal ein Fenster früher als die Baselines und zweimal im selben Fenster. Die Angaben beschreiben die beobachtete Ausgabe- und Schwellenreihenfolge. Sie belegen weder eine höhere ereignisbezogene Erkennungsquote noch eine kürzere absolute Verzögerung ab physikalischem Beginn.

8.2 Schwellen, Alarmhäufigkeit und faire Bewertung

Die Autoencoder-MSE gehört zum standardisierten Eingaberaum und zur Rekonstruktionsfehlerverteilung des trainierten Modells. Ein hoher Score zeigt eine Abweichung von dieser Referenz; er identifiziert keine bestimmte physikalische Ursache. Beabsichtigte Anregung, veränderte Orientierung und Änderungen normaler Betriebsbedingungen können ähnliche Auswirkungen haben. Das Beibehalten der eingefrorenen Schwellen sichert den Vergleich innerhalb einer Reihe; eine spätere Rekalibrierung wäre eine eigenständige Untersuchung.

Die Normierung Score/Schwelle macht den Grenzübertritt sichtbar, gleicht aber die Verteilungen der drei Methoden nicht an. Ein Verhältnis von 3 beim Autoencoder ist nicht dreimal so viel physische Störung wie ein Verhältnis von 1 beim Isolation Forest. Die gemeinsame P99-Regel ist ein nachvollziehbarer Kalibrierungsansatz, garantiert aber bei kleinen und zeitlich abhängigen Normaldaten keine identische Sensitivität oder Spezifität im Test.

Die Kalibrierung ist eine wesentliche Unsicherheitsquelle des frühen Alarmvergleichs. Nur eine Normalaufnahme liefert die 97 P99-Scores, und dieselbe Aufnahme dient der Modellüberwachung. Besonders beim Isolation Forest in Fenster 108 liegt der Score knapp oberhalb der Grenze. Eine andere unabhängige Kalibrierungsstichprobe könnte die relative Alarmreihenfolge verändern. Dies ist eine plausible Unsicherheit, keine im vorliegenden Datensatz nachgewiesene Ursache. Für eine faire Bewertung bei vergleichbarer Spezifität wäre zusätzlich eine unabhängige Schätzung der Fehlalarmraten erforderlich.

Mehr Alarmfenster können längere Empfindlichkeit, Nachschwingen oder Fehlalarme bedeuten. Ohne unabhängige Ereignisreferenz lässt sich dies nicht vollständig trennen. Ebenso ist „kein Alarm außerhalb der Bedienmarker“ ein nützlicher Kontrollbefund, aber kein statistischer Nachweis einer Fehlalarmrate von null. Alarmfenster desselben Anrufs dürfen nicht als unabhängige erfolgreiche Wiederholungen gezählt werden.

8.3 Vergleichbarkeit und interne Gültigkeit

Der gemeinsame Live-Lauf ist für die Rangfolge im konkreten Ereignis am aussagekräftigsten, weil alle Methoden dieselben Rohfenster erhalten. Er beantwortet jedoch nicht den isolierten Ressourcenbedarf jedes Verfahrens. Die Einzelprozesse helfen bei dieser zweiten Frage, sind aber aufgrund unterschiedlicher Eingaben, Laufdauern und fehlender Wiederholungsmatrix nur deskriptiv vergleichbar. Replay trennt die Wirkung der Eingabedaten von unterschiedlichen Anrufen, ersetzt aber keine physikalische Wiederholung.

Weitere Unsicherheit entsteht durch die Kopplung Smartphone-Tisch-Aufbau-Sensor. Position, Unterlage und Vibrationsmuster wurden nicht unabhängig quantifiziert. Das aktuelle Normalprofil ist setupbezogen; eine Änderung des mechanischen Wegs kann bereits die Normalverteilung verschieben. Die fehlende vollständige Bestätigung der finalen Smartphonegeometrie und der Entwicklungsstatus der Messkette werden deshalb nicht durch einen positiven Einzelbefund aufgehoben.

Nominelle Sensorrate, Host-Zeitstempel und physikalischer Ereignisbeginn sind unterschiedliche Größen. Die zeitliche Lage einer Anregung innerhalb des rund 0,617-s-Fensters beeinflusst, wann ein auffälliges vollständiges Fenster verfügbar wird. Eine kurze Modellberechnung garantiert deshalb keine ebenso kurze Ende-zu-Ende-Erkennung. Gleichzeitige Worker-Starts beseitigen weder Betriebssystem-Scheduling noch Konkurrenz in Python und nativen Bibliotheken. Die 811 Rohfenster der ausgewählten Aufnahmen repräsentieren vier physikalische Ereignisse; die zusätzlichen Replay-Entscheidungen erhöhen diese Ereigniszahl nicht.

8.4 Raspberry-Pi-Bedarf und Grenzen der Übertragung

Im beobachteten Einzelbetrieb benötigt RMS den geringsten Rechenaufwand und Prozessspeicher, Isolation Forest den höchsten. Der Autoencoder liegt dazwischen. Die Rangfolge gilt für die instrumentierten Prozesse und Laufdauern. Die Verfahrenswahl erfordert zusätzlich eine belastbare Erkennungsbewertung und abgesicherte Messqualität.

Die gemeinsame Ausführung hält die beobachteten Fensterfristen ein. Dies garantiert weder Dauerbetrieb noch Eignung unter anderer Systemlast. Der Raspberry Pi 5 besitzt wesentlich mehr Ressourcen als ein typischer Mikrocontroller; eine Übertragung auf STM32-Hardware erfordert Portierung und erneute Messung.

9 Ausblick

Für eine belastbare Fortsetzung ist zuerst die physische Referenz zu verbessern. Vibrationsbeginn und -ende sollten mit einer unabhängigen, zeitlich zugeordneten Referenz erfasst werden. Smartphoneposition, Auflage und Anregungsmuster sind fest zu dokumentieren. Ein automatisiertes reproduzierbares Vibrationssignal wäre für Wiederholungen geeigneter als nur die manuelle Bestätigung eines Anrufs. Es bleibt dabei zu prüfen, welche reale Fehlerklasse durch eine äußere Tischanregung tatsächlich repräsentiert wird.

Danach sind mehrere unabhängige Ereignisse und längere Normalaufnahmen mit unverändert eingefrorenen Modellen nötig. Die Auswertung sollte vorab festlegen, wie ein Ereignis als erkannt gilt, wie Nachschwingen behandelt wird und welche Zeitabweichung akzeptiert wird. Eine Train-/Validierungs-/Testtrennung nach vollständigen Aufnahmen bleibt notwendig. Erst mit solchen Daten sind Erkennungsquote, Fehlalarmrate und Erkennungsverzögerung mit sinnvoller Unsicherheit vergleichbar.

Für den Raspberry-Pi-Vergleich sollten alle Methoden wiederholt auf denselben gespeicherten Daten in getrennten Prozessen laufen, ergänzt um gemeinsame Live-Versuche. Reihenfolge, Warmup, Messdauer, CPU-Konfiguration und Hintergrundlast sind festzulegen beziehungsweise zu rotieren. GUI-Last und Modellstart sind bei Bedarf getrennt zu messen. Elektrische Leistung erfordert eine zusätzliche Spannungs-/Strommessung mit klarer Systemgrenze; erst dann sind Energie pro Fenster oder Energie pro erkanntem Ereignis sinnvoll bestimmbar.

Eine Fortsetzung sollte alle drei Verfahren mit vorab festgelegten Endpunkten, unabhängiger Ereignisreferenz und zusätzlichen Normalaufnahmen vergleichen. Dabei sind Modellwahl, Schwellenkalibrierung und Test möglichst auf getrennte vollständige Aufnahmen zu verteilen. Die Wahl einer Methode für den Einsatz ergibt sich aus Erkennungsqualität, Fehlalarmen, Zeitverhalten und Ressourcenbedarf. Die Untersuchung eines bevorzugten Modells muss ebenso negative wie positive Ergebnisse zulassen.

10 Fazit

Die Arbeit realisiert eine lokal ausführbare und nachvollziehbar instrumentierte Vergleichspipeline für RMS, Isolation Forest und einen Float32-TFLite-Autoencoder auf dem Raspberry Pi 5. Die explorative Smartphoneuntersuchung verbindet Einzelbetrieb, Replay und gemeinsamen Live-Betrieb. Identische Rohfenster, eingefrorene Artefakte und reproduzierte Entscheidungen ermöglichen einen transparenten Vergleich. Datenerfassung und Inferenz funktionieren im untersuchten Umfang; vollständige Messqualität und langfristige Wiederholbarkeit bleiben teilweise offen.

Für FF2 zeigt sich im gemeinsamen Ereignis ein früherer AE-Ausgabezeitpunkt: etwa 19,1 ms gegenüber IF bei gleichem ersten Alarmfenster 108 und 615,7 ms gegenüber RMS, das erst Fenster 109 markiert. Im Replay der drei ausgewählten Einzelereignisse beginnt der AE einmal früher und zweimal gleichzeitig mit den Baselines. Diese Beobachtungen beschreiben den untersuchten Datenstand. Ohne unabhängige physikalische Ereignislabels lassen sich weder F1 und bestätigte Fehlalarmrate noch die absolute Erkennungsverzögerung bestimmen.

Für FF3 erreicht der gemeinsame Live-Prozess einen CPU-Median von 13,04 % eines logischen Kerns und maximal 174,42 MiB abgetasteten RSS. Je Methode liegen alle 196 überprüfbaren Entscheidungen vor der nächsten Fensterankunft. Auch das P99-Latenzziel wird im beobachteten Lauf erfüllt. RMS benötigt den geringsten berechnungsbezogenen Aufwand; Isolation Forest den höchsten in den ausgewählten Einzelprozessen. Elektrischer Energiebedarf, andere Hardwareplattformen und Dauerbetrieb wurden nicht nachgewiesen.

FF1 wird durch die begründete Systemauswahl und ihre dokumentierte Umsetzung beantwortet. Für FF4 belegen Journale, Hashes und Replay die technische Vergleichbarkeit auf identischen Eingaben; die kleine Kalibrierungsbasis, unvollständig vermessene Anregung und geringe Zahl unabhängiger Ereignisse begrenzen weitergehende Aussagen. Der wissenschaftliche Beitrag liegt in der überprüfbaren Trennung von Alarmverhalten, Ausgabezeitpunkt und technischem Aufwand. Eine allgemeine Überlegenheit oder zuverlässige Defekterkennung ist daraus nicht ableitbar.

Anhang

Digitale Belege und Fassungszuordnung

Die digitalen Belege beziehen sich auf das Repository Thomas-mu/MA, Branch main, Commit fa5cbd906d2d562a2dd12f1d31d617e6cd44b0dc vom 19. September 2026. Zitierbarer Stand:

<https://github.com/Thomas-mu/MA/tree/fa5cbd906d2d562a2dd12f1d31d617e6cd44b0dc>.

Tabelle A-1 ordnet die Smartphoneaufnahmen, das Normalprofil und die Implementierung zu. Die Pfade sind relativ zum Repositorystamm zu lesen. Die zusätzlich berichteten Fristzählungen sind aus B1/joint_trials/all_methods_01/decisions.csv abgeleitet. Der Commit fixiert Daten- und Quellcodestand, nicht spätere redaktionelle Änderungen dieses Berichts.

Tabelle A-1: Nachweisverzeichnis

ID	Inhalt	Projektpfad
B1	Smartphoneversuche: Bündel, Einzelaufnahmen, Replay und gemeinsamer Live-Lauf	results/phone_vibration_pilot_20260919/
B2	Normalprofil, Aufnahmetrennung, Trainings- und Exportnachweise	profiles/standlauf_pwm75_200hz/
B3	Einzel- und gemeinsamer Smartphone-Recorder	src/phone_vibration_experiment.py src/joint_phone_vibration_experiment.py
B4	Gemeinsame Auswertung, Training und Sensortreiber	src/common_comparison.py src/calibrate_and_train.py src/train_clean_autoencoder.py src/train_autoencoder.py src/adxl345.py

Das eingefrorene Bündel der Smartphoneuntersuchung besitzt den in Laufdateien und Replay zugeordneten SHA-256-Wert

dbfad8529b642d1e4c74119b3865b0600c3a7668198328e1112707c7502b2eb1. Der SHA-256-Wert der gemeinsamen Rohdatei lautet

241d333071ad320bc2f1282b5722a30f54fd06ae968ecc2a9250056eb93a1074. Die vollständigen Artefaktmanifeste liegen in B1. Hashes sichern die Zuordnung von Dateien und Modellständen; sie bestätigen nicht die physikalische Gültigkeit der Messanordnung.

Die Rohdaten und eingefrorenen Modellartefakte bilden die Primärbelege der eigenen Messungen. Daraus erzeugte Tabellen, Abbildungen und Präsentationen sind abgeleitete Darstellungen und keine zusätzlichen Versuche. Angaben aus Herstellerunterlagen, wissenschaftlicher Literatur und Softwaredokumentation sind davon getrennt im Literaturverzeichnis nachgewiesen.

Umgebung der dokumentierten Ausführung: Raspberry Pi 5 mit vier logischen Kernen, 64-Bit-Linux und Python 3.13.5. Die Laufdateien der Smartphoneaufnahme nennen NumPy

2.5.1, scikit-learn 1.9.0, ai-edge-litert 2.1.6 und psutil 7.2.2. Der Trainingsprofil-Snapshot stammt aus einer separat erfassten Umgebung und nennt unter anderem NumPy 2.5.2 und TensorFlow 2.21.0. Die jeweilige Provenienzdatei hat Vorrang vor einer pauschalen Gleichsetzung aller Softwarestände. Die Hardwareauswahl ist davon unberührt.

Lesen der aktuellen Messdateien

raw.csv enthält sample_index, window_index, monotone und UTC-Hostzeit, x_g, y_g, z_g und Qualitätsmerkmale. Ein vollständiges Modellfenster besteht aus 128 aufeinanderfolgenden XYZ-Punkten desselben Laufs. Ausgewertet werden die vollständigen Fenster der gesamten nach Warmup gestarteten Aufnahme. Restpunkte bleiben dokumentiert und werden nicht durch künstliche Werte zu zusätzlichen Fenstern ergänzt.

decisions.csv ordnet Score, eingefrorene Schwelle, Entscheidung und Fehlerstatus dem Fenster zu. Im gemeinsamen Lauf kommen method, input_sha256, dispatch_ns, worker_start_ns und decision_ns hinzu. window_to_decision_ms verwendet den Host-Abschluss des letzten Fensterpunkts. resources.csv gehört zum gemeinsamen Prozess, nicht zu einer isolierten Methode. CPU-Prozente, RSS, Temperatur und Takt sind Messproben des beobachteten Prozesses und Systems.

cues.jsonl speichert die Bedienereignisse; run.json beschreibt Laufstatus, Sensor, PWM und Erfassungsintegrität. Der JSON-Wert null oder ein fehlender Eintrag für true_detection_delay_ms, detection_rate beziehungsweise false_alarm_rate kennzeichnet einen nicht bestimmbaren Wert; daraus dürfen nicht 0 ms oder 0 % abgeleitet werden. Die Zahl 0 ist dagegen ein numerischer Wert und kein Platzhalter für fehlende Daten. UTC-Einträge sind keine nachträglich erfundenen physikalischen Ereignisgrenzen. Bilder und Präsentationsansichten sind Ergebnisdarstellungen, keine zusätzlichen Messungen.

KI-Unterstützung und fachliche Verantwortung

Codex von OpenAI unterstützte Softwareentwicklung, Fehlersuche, Auswertung, Grafikaufbereitung, Quellenprüfung sowie Formulierung, Überarbeitung und Formatprüfung einschließlich generierter Textpassagen. Physische Einrichtung und protokollierte Sichtbeobachtungen beruhen auf Angaben des Versuchsverantwortlichen. KI-Ausgaben ersetzen weder wissenschaftliche Quellen noch Messnachweise.

Die fachliche Verantwortung für Versuchsangaben, Quellenverwendung und Schlussfolgerungen sowie die wahrheitsgemäße Angabe der Hilfsmittel verbleiben beim Verfasser. Die hier vorgenommene Eingrenzung auf den Smartphoneversuch ist eine redaktionelle Festlegung des Untersuchungsumfangs; sie macht die Evaluation nicht zu einer vorab registrierten Studie.

Literaturverzeichnis

- Analog Devices (2022). ADXL345: 3-Axis, ± 2 g/ ± 4 g/ ± 16 g Digital Accelerometer. Datenblatt, Revision G, Mai 2022. <https://www.analog.com/media/en/technical-documentation/data-sheets/adxl345.pdf> (Abruf: 12.09.2026).
- Analog Devices (2025). ADXL354/ADXL355: Low Noise, Low Drift, Low Power, 3-Axis MEMS Accelerometers. Datenblatt, Revision D, Juni 2025. https://www.analog.com/media/en/technical-documentation/data-sheets/adxl354_adxl355.pdf (Abruf: 12.09.2026).
- Antonini, M., Pincheira, M., Vecchio, M., & Antonelli, F. (2023). An Adaptable and Unsupervised TinyML Anomaly Detection System for Extreme Industrial Environments. *Sensors*, 23(4), 2344. <https://doi.org/10.3390/s23042344> (Abruf: 12.09.2026).
- ARCTIC (o. J.). Specifications: P12 Pro PST. Technisches Datenblatt. https://www.arctic.de/media/2c/de/c6/1750758983/Spec_Sheet_P12_Pro_PST_EN.pdf (Abruf: 12.09.2026).
- Buttazzo, G. C. (2011). *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications* (3. Aufl.). Springer. <https://doi.org/10.1007/978-1-4614-0676-1> (Abruf: 12.09.2026).
- Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly Detection: A Survey. *ACM Computing Surveys*, 41(3), Artikel 15, 1–58. <https://doi.org/10.1145/1541880.1541882> (Abruf: 12.09.2026).
- Department for Communities and Local Government (2009). *Multi-criteria analysis: a manual*. London. ISBN 978-1-4098-1023-0. <https://www.gov.uk/government/publications/multi-criteria-analysis-manual-for-making-government-policy> (Abruf: 12.09.2026).
- Garay, C. E., Miranda Bonomi, F. A., Mansilla, G. N., Fagre, M., Guzmán, S. G., Ritorto, P. A., Perez, F. I., & Katz, M. (2026). A Multimodal TinyML-Based Predictive Maintenance Architecture for Industrial IoT in the 6G Era. *Sensors*, 26(14), 4536. <https://doi.org/10.3390/s26144536> (Abruf: 19.09.2026).
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Kapitel 14: Autoencoders. <https://www.deeplearningbook.org/contents/autoencoders.html> (Abruf: 19.09.2026).
- Google (2026a). Convert TensorFlow models. LiteRT-Dokumentation, aktualisiert am 28.05.2026. https://developers.google.com/edge/litert/conversion/tensorflow/convert_tf (Abruf: 19.09.2026).
- Google (2026b). Post-training quantization. LiteRT-Dokumentation, aktualisiert am 28.05.2026. https://developers.google.com/edge/litert/conversion/tensorflow/quantization/post_training_quantization (Abruf: 19.09.2026).
- Google (2026c). Quickstart for Linux-based devices with Python. LiteRT-Dokumentation, aktualisiert am 28.05.2026. <https://developers.google.com/edge/litert/microcontrollers/python> (Abruf: 19.09.2026).
- Google (2026d). Install TensorFlow with pip. TensorFlow-Dokumentation, aktualisiert am 12.03.2026. <https://www.tensorflow.org/install/pip> (Abruf: 19.09.2026).
- Jardine, A. K. S., Lin, D., & Banjevic, D. (2006). A review on machinery diagnostics and prognostics implementing condition-based maintenance. *Mechanical Systems and Signal Processing*, 20(7), 1483–1510. <https://doi.org/10.1016/j.ymssp.2005.09.012> (Abruf: 12.09.2026).
- Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), Artikel 15. <https://doi.org/10.1145/2382577.2382579> (Abruf: 12.09.2026).
- Knap, P., Jachymczyk, U., & Lalik, K. (2026). Leakage-Safe, Reproducible Benchmarking for Vibration-Based Fault Diagnosis. *PHM Society European Conference*, 9(1), 1–8. <https://doi.org/10.36001/phme.2026.v9i1.4924> (Abruf: 12.09.2026).

- Lessmeier, C., Kimotho, J. K., Zimmer, D., & Sextro, W. (2016). Condition Monitoring of Bearing Damage in Electromechanical Drive Systems by Using Motor Current Signals of Electric Motors: A Benchmark Data Set for Data-Driven Classification. PHM Society European Conference, 3(1). <https://doi.org/10.36001/phme.2016.v3i1.1577>.
- Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. Proceedings of the Eighth IEEE International Conference on Data Mining, 413–422. <https://doi.org/10.1109/ICDM.2008.17> (Abruf: 12.09.2026).
- NASA (2023). 6.8 Decision Analysis. Webfassung des NASA Systems Engineering Handbook. <https://www.nasa.gov/reference/6-8-decision-analysis/> (Abruf: 12.09.2026).
- NVIDIA (o. J.). Jetson Orin Nano Super Developer Kit. Technische Spezifikationen. <https://www.nvidia.com/de-de/autonomous-machines/embedded-systems/jetson-orin/nano-super-developer-kit/> (Abruf: 12.09.2026).
- ONNX Runtime developers (o. J.). Python. ONNX Runtime-Dokumentation zur Installation und Ausführung. <https://onnxruntime.ai/docs/get-started/with-python.html> (Abruf: 12.09.2026).
- Oriental Motor (o. J.). Servo Motor Features Overview. <https://www.orientalmotor.com/servo-motors/technology/servo-motor-features.html> (Abruf: 12.09.2026).
- PCB Piezotronics (o. J.). Model 356A32: Triaxial Accelerometer. Technische Spezifikationen. <https://www.pcb.com/products?m=356a32> (Abruf: 12.09.2026).
- Pittino, F., Puggl, M., Moldaschl, T., & Hirschl, C. (2020). Automatic Anomaly Detection on In-Production Manufacturing Machines Using Statistical Learning Methods. *Sensors*, 20(8), 2344. <https://doi.org/10.3390/s20082344> (Abruf: 12.09.2026).
- PyTorch (o. J.). ExecuTorch Documentation. Dokumentation zum Modellexport und zur lokalen Ausführung. <https://docs.pytorch.org/executorch/stable/index.html> (Abruf: 12.09.2026).
- Randall, R. B., & Antoni, J. (2011). Rolling element bearing diagnostics—A tutorial. *Mechanical Systems and Signal Processing*, 25(2), 485–520. <https://doi.org/10.1016/j.ymssp.2010.07.017> (Abruf: 12.09.2026).
- Raspberry Pi Ltd. (2026a). Raspberry Pi 4 Model B. Product Brief, April 2026. <https://pip-assets.raspberrypi.com/categories/545-raspberry-pi-4-model-b/documents/RP-008344-DS-1-raspberry-pi-4-product-brief> (Abruf: 19.09.2026).
- Raspberry Pi Ltd. (2026b). Raspberry Pi 5. Product Brief, April 2026. <https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf> (Abruf: 19.09.2026).
- Ren, H., Anicic, D., & Runkler, T. A. (2021). TinyOL: TinyML with Online-Learning on Microcontrollers. *International Joint Conference on Neural Networks*, 1–8. <https://doi.org/10.1109/IJCNN52387.2021.9533927> (Abruf: 12.09.2026).
- Ruff, L., Kauffmann, J. R., Vandermeulen, R. A., Montavon, G., Samek, W., Kloft, M., Dietterich, T. G., & Müller, K.-R. (2021). A Unifying Review of Deep and Shallow Anomaly Detection. *Proceedings of the IEEE*, 109(5), 756–795. <https://doi.org/10.1109/JPROC.2021.3052449> (Abruf: 12.09.2026).
- scikit-learn developers (o. J. a). Common pitfalls and recommended practices. Benutzerhandbuch. https://scikit-learn.org/stable/common_pitfalls.html (Abruf: 12.09.2026).
- scikit-learn developers (o. J. b). Cross-validation: evaluating estimator performance. Insbesondere Aufteilung gruppiertter Daten. https://scikit-learn.org/stable/modules/cross_validation.html (Abruf: 12.09.2026).
- scikit-learn developers (o. J. c). Metrics and scoring: quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html (Abruf: 12.09.2026).

- scikit-learn developers (o. J. d). IsolationForest. API-Dokumentation.
<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html> (Abruf: 12.09.2026).
- SciPy developers (o. J. a). Discrete Fourier Transforms (scipy.fft). SciPy-Benutzerhandbuch.
<https://docs.scipy.org/doc/scipy/tutorial/fft.html> (Abruf: 12.09.2026).
- SciPy developers (o. J. b). Signal Processing (scipy.signal), Abschnitt Spectral Analysis. SciPy-Benutzerhandbuch. <https://docs.scipy.org/doc/scipy/tutorial/signal.html#spectral-analysis> (Abruf: 12.09.2026).
- Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). Edge Computing: Vision and Challenges. IEEE Internet of Things Journal, 3(5), 637–646. <https://doi.org/10.1109/IIOT.2016.2579198> (Abruf: 12.09.2026).
- STMicroelectronics (2026). STM32H742xl/G STM32H743xl/G: 32-bit Arm Cortex-M7 480 MHz MCUs, up to 2 MB flash, up to 1 MB RAM, 46 com. and analog interfaces. Datenblatt DS12110, Revision 11, Januar 2026. <https://www.st.com/resource/en/datasheet/stm32h743vi.pdf> (Abruf: 19.09.2026).
- Universität Paderborn, Lehrstuhl Konstruktions- und Antriebstechnik (o. J.). Bearing Data Center. Beschreibung des Lagerprüfstands und des Datensatzes.
<https://mb.uni-paderborn.de/en/kat/research/bearing-datacenter> (Abruf: 12.09.2026).